

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HCM

KHOA CÔNG NGHỆ ĐIỆN TỬ

-----❧-----



DỰ ÁN KỸ THUẬT NÂNG CAO

**HỆ THỐNG ĐO KHOẢNG CÁCH BẰNG CẢM
BIẾN SIÊU ÂM US 100 GIAO TIẾP UART VỚI
ESP32 S3 VÀ HIỂN THỊ BẰNG PHẦN MỀM
WINFORM C#**

GVHD: ThS. Nguyễn Duy Khanh

SVTH: Đinh Hữu Triết

Lê Gia Bảo

Lê Huỳnh Nhật Phi

LỚP: ĐHĐTMT18B

TP. HCM, tháng 11-2025

THÔNG TIN CHUNG

Họ và tên sinh viên : Đinh Hữu Triết MSHV: 22703351

Họ và tên sinh viên : Lê Gia Bảo MSHV: 22705601

Họ và tên sinh viên : Lê Huỳnh Nhật Phi MSHV: 22723001

Lớp : DHDTMT18B Khóa: K18

Chuyên ngành : Kỹ thuật máy tính Mã chuyên ngành: 7480108

SĐT : 0345897689

Email : huutriet.0976@gmail.com

Địa chỉ liên hệ : 140/11/16/12 Vườn Lài khu phố 2 quận 12

Tên đề tài : Hệ thống đo khoảng cách bằng cảm biến siêu âm giao tiếp Uart với ESP32
S3 và hiển thị bằng phần mềm Winform C#

Người hướng dẫn : Ths. Nguyễn Duy Khanh

Tp. Hồ Chí Minh, ngày tháng năm 2025

Người hướng dẫn

(Ký và ghi rõ họ tên)

Sinh viên

(Ký và ghi rõ họ tên)

This image shows a blank sheet of white paper with horizontal dashed lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no other markings or text on the page.

Giáo viên hướng dẫn

NHẬN XÉT GIÁO VIÊN PHẢN BIỆN

Tp.HCM, ngày tháng năm 2025

Giáo viên phản biện

LỜI CẢM ƠN

Để hoàn thành đồ án môn học này, chúng em đã nhận được sự hỗ trợ giúp đỡ tận tình của các thầy cô, người thân và bạn bè trong suốt thời gian thực hiện đề tài .

Chúng em xin chân thành cảm ơn quý thầy/cô khoa Công nghệ Điện tử, trường Đại học Công nghiệp TP.HCM đã tận tình giảng dạy, cung cấp cho chúng em những kiến thức quý báu trong những học kì vừa qua. Đặc biệt, em xin trân trọng gửi lời cảm ơn sâu sắc đến thầy Nguyễn Duy Khanh, người đã trực tiếp hướng dẫn, giúp đỡ nhóm chúng em trong suốt thời gian thực hiện đề tài của môn học này.

Chúng em xin chân thành cảm ơn đến gia đình, bạn bè là những người luôn đồng hành động viên giúp đỡ về vật chất và tinh thần để chúng em có đủ tự tin hoàn thành đồ án này.

Chúng em xin chân thành cảm ơn.

Tp.HCM, ngày tháng năm 2025

Nhóm sinh viên thực hiện

Đinh Hữu Triết

Lê Gia Bảo

Lê Huỳnh Nhật Phi

MỤC LỤC

LỜI CẢM ƠN.....	I
CÁC TỪ VIẾT TẮT	IV
LỜI MỞ ĐẦU	2
1.ĐẶT VẤN ĐỀ	2
2.MỤC TIÊU CHÍNH CỦA ĐỀ TÀI	2
3.ĐỐI TƯỢNG NGHIÊN CỨU VÀ PHẠM VI.....	3
4.Ý NGHĨA THỰC TUYÊN CỦA ĐỀ TÀI.....	3
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI.....	4
1.1 BỐI CẢNH NGHIÊN CỨU.....	4
1.2 TÓM TẮT CÁC NGHIÊN CỨU LIÊN QUAN ĐÃ CÓ VÀ KINH NGHIỆM RÚT RA.....	4
1.3 NHỮNG KHÓ KHĂN VÀ VẤN ĐỀ CHƯA ĐƯỢC GIẢI QUYẾT.....	5
CHƯƠNG 2:CƠ SỞ LÝ THUYẾT	6
2.1 TỔNG QUAN VỀ CÁC LINH KIỆN CHỦ ĐẠO	6
2.1 Cảm biến US100.....	6
2.2 SMD LM117-3.3V.....	8
2.3 Linh kiện dán SMD ADuM1201ARZ – Bộ cách ly số hai kênh...9	
2.4 Vi xử lý ESP32-S3 WROOM-1	10
2.5 CP2102 Module (USB to TTL).....	13
2.6 TỔNG QUAN VỀ PHẦN MỀM, GIAO THỨC UART, PHƯƠNG PHÁP ĐO VÀ THUẬT TOÁN	15
2.6.1 Phần mềm WinForms C#	15
2.6.2 Phần mềm lập trình Firmware (VS Code + PlatformIO).....	15
2.6.3 Cơ sở lý thuyết về thuật toán và phương pháp xử lý dữ liệu	16
2.6.4 Thuật toán kiểm tra lỗi (CRC).....	17
2.6.5 Giao thức UART	18
2.6.6 UART cứng là gì	19
2.6.7 UART mềm là gì	19
2.6.8 Bản so sánh UART cứng và UART mềm.....	20

2.6.9 Phương pháp đo điện áp và dòng tiêu thụ	20
2.6.10 Phương pháp đo và quan đánh giá tín hiệu Waveform bằng máy hiện sóng	21
2.6.11 Phương pháp đánh giá phần mềm	21
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	22
3.1 PHÂN TÍCH THIẾT KẾ HỆ THỐNG	22
3.1.1 Phân tích, xác định thông số đầu vào đầu ra của hệ thống	22
3.1.2 Thiết kế sơ đồ khối	22
3.1.3 Thiết kế nguyên lý	24
3.1.4 Thiết kế bộ hạ áp 5V to 3.3V	25
3.1.5 Thiết kế Opto chống nhiễu Uart	26
3.1.6 Thiết kế khối Enabl và Boot	27
3.2 THI CÔNG	28
3.2.1 Vẽ mạch in	28
3.3 THIẾT KẾ GIẢI THUẬT VÀ GIAO DIỆN	30
3.3.1 Lưu đồ giải thuật Firmware	30
3.3.2 Thiết kế giao diện người dùng và lưu đồ giải thuật Winform C#	
37	
CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	46
4.1 GIỚI THIỆU MỤC TIÊU CHÍNH CỦA THỰC NGHIỆM	46
4.1.1 Mục tiêu chính của thực nghiệm	46
4.2 KIỂM TRA VÀ ĐO ĐẶC PHẦN CỨNG	47
4.2.1 Đo nguồn, dòng tiêu thụ và đánh giá	47
4.2.2 Đo Waveform UART mềm và đánh giá	49
4.2.3 Đo Waveform UART US100 và đánh giá	53
4.2.4 Thực nghiệm phần mềm và đánh giá	57
4.2.5 Minh chứng kiểm tra thực tế và đánh giá khái quát.....	58
4.3 LẮP RÁP MÔ HÌNH.....	59
CHƯƠNG 5: KẾT LUẬN VÀ ĐỀ XUẤT PHÁT TRIỂN	61
TÀI LIỆU THAM KHẢO	62

CÁC TỪ VIẾT TẮT

Từ viết tắt	Tên tiếng anh
CRC	Cyclic Redundancy Check
D2C	Device-to-Cloud
D2D	Device-to-Device
GND	Ground
GUI	Graphical User Interface
IC	Integrated Circuit
LED	Light Emitting Diode
MCU	Microcontroller Unit
PCB	Printed Circuit Board
RX	Receive
SMD	Surface-Mount Device
SoC	System on a Chip
TX	Transmit
UART	Universal Asynchronous Receiver/Transmitter
VCC	Power

LỜI MỞ ĐẦU

1. Đặt vấn đề

Trong xu thế phát triển mạnh mẽ của công nghệ đặc biệt là công nghệ cao, việc đo khoảng cách chính xác đóng vai trò quan trọng trong nhiều ứng dụng: từ robot tránh vật cản, giám sát mức nước, cho đến hệ thống cảnh báo an toàn. Cảm biến US-100 nổi bật hơn nhờ khả năng đo ổn định và đặc biệt là hỗ trợ giao tiếp UART trực tiếp. Với chế độ này, cảm biến tự động tính toán khoảng cách và gửi dữ liệu qua giao tiếp nối tiếp đến vi điều khiển, chính vì điều này đã góp phần tăng độ chính xác và tin cậy cho hệ thống.

Đề khai thác tối đa ưu điểm này, đề tài lựa chọn vi điều khiển ESP32-S3 – một nền tảng mạnh mẽ, tích hợp nhiều cổng UART phần cứng, tốc độ xử lý cao, đồng thời có khả năng mở rộng sang các ứng dụng IoT. Trong hệ thống, ESP32-S3 sẽ nhận dữ liệu từ US-100 qua UART mặc định, sau đó truyền dữ liệu sang máy tính bằng UART mềm (software UART) do chúng em thiết kế tự xây dựng. Chúng em nhận định rằng đây là một điểm khác biệt quan trọng, thể hiện khả năng thiết kế giao thức mà tại đó truyền thông có thể tùy chỉnh được, không phụ thuộc hoàn toàn vào phần cứng.

Song song với phần cứng, việc phát triển phần mềm hiển thị bằng WinForm C# trên máy tính mang lại giao diện trực quan, dễ sử dụng, cho phép người dùng theo dõi dữ liệu đo khoảng cách theo thời gian thực. Nhờ đó, hệ thống không chỉ phục vụ mục đích nghiên cứu mà còn có thể ứng dụng trong thực tế.

Chính vì những lý do trên, đề tài “*Hệ thống đo khoảng cách bằng cảm biến siêu âm US-100 giao tiếp UART với ESP32-S3 và hiển thị bằng phần mềm WinForm C#*” mang tính cấp thiết, phù hợp với nhu cầu thực tiễn, và có ý nghĩa trong việc nâng cao khả năng ứng dụng công nghệ cao vào đời sống và sản xuất.

2. Mục tiêu chính của đề tài

Đề tài hướng đến việc xây dựng một hệ thống đo khoảng cách hoàn chỉnh, kết hợp từ phần cứng đến phần mềm, với các mục tiêu cụ thể được đề ra như sau:

- Thiết kế và triển khai kết nối cảm biến US-100 với ESP32-S3 qua giao thức UART.
- Lập trình chương trình trên ESP32-S3 để đọc dữ liệu khoảng cách và truyền về máy tính(tạo uart mềm).
- Xây dựng ứng dụng WinForm C# hiển thị dữ liệu đo khoảng cách theo thời gian thực, có Hàm Test độ chính xác Uart.

- Đánh giá độ chính xác được độ ổn định và hiệu quả của hệ thống trong các thử nghiệm
- Đề xuất cải tiến và khả năng mở rộng cho đề tài .

3.Đối tượng nghiên cứu và phạm vi

a. Đối tượng nghiên cứu:

- Vi điều khiển ESP32-S3.
- Cảm biến siêu âm US-100.
- Giao thức truyền thông chính UART (tốc độ 9600)
- Phần mềm WinForm C# trên máy tính giao tiếp với ESP32 S3 thông qua Uart mềm với tốc độ 9600

b.Phạm vi nghiên cứu:

- Khoảng cách đo từ 2 cm đến 100 cm theo quy ước của nhóm
- Môi trường thử nghiệm trong nhà, phòng học
- Thời gian nghiên cứu và thực nghiệm từ tháng 9 tới tháng 11 năm 2025
- Hệ thống của chúng em tập trung chính vào việc đo và hiển thị khoảng cách, lập trình Uart mềm giao tiếp với Winform C#, có hàm Test tốc độ Uart mềm, chưa mở rộng thêm phần cơ sở dữ liệu, chưa mở rộng thêm phần truyền dữ liệu qua mạng.

4.Ý nghĩa thực tiễn của đề tài

Đề tài cung cấp rõ các ý nghĩa thực tiễn như sau:

- Ứng dụng thực tế: Hệ thống có thể áp dụng trong giám sát mức nước trong ao, hồ hay bồn chứa nước dân dụng , robot tránh vật cản , xe tự hành
- Giải quyết vấn đề: Đề tài đã mang lại giải pháp đo khoảng cách chi phí thấp, dễ triển khai, trực quan và dễ dàng sử dụng
- Hiệu quả: Góp phần nâng cao độ chính xác, độ tin cậy trong đo đạc, đồng thời tạo tiềm năng nền tảng để mở rộng sang các ứng dụng IoT nhằm phục vụ những hộ gia đình, trang trại, công ty quy mô nhỏ

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1 Bối cảnh nghiên cứu

Hiện nay trong lĩnh vực đo khoảng cách và cần độ chính xác mang tính tương đối, cảm biến siêu âm được sử dụng rộng rãi nhờ giá thành thấp, dễ triển, cộng đồng sử dụng nhiều đánh giá nhận được rất tích cực. Các loại cảm biến truyền thống phổ biến khác như HC-SR04 thường yêu cầu vi điều khiển đo thời gian phản hồi tín hiệu (echo), dễ bị nhiễu và phụ thuộc nhiều vào độ chính xác của xung.

Cảm biến US-100 khắc phục một phần hạn chế này khi hỗ trợ chế độ UART mặc định, cho phép cảm biến tự động tính toán khoảng cách và gửi dữ liệu trực tiếp qua giao tiếp nối tiếp. Điều này giúp giảm tải cho vi điều khiển, đồng thời tăng độ ổn định và độ chính xác.

Vi điều khiển ESP32-S3 là nền tảng mạnh mẽ, có nhiều cổng UART phân cứng, tốc độ xử lý cao và tích hợp Wi-Fi/Bluetooth. Tuy nhiên, trong đề tài này, ngoài việc sử dụng UART mặc định để nhận dữ liệu từ US-100, hệ thống còn yêu cầu **tự xây dựng UART mềm** để truyền dữ liệu từ ESP32-S3 sang ứng dụng WinForm C#. Đây là điểm khác biệt quan trọng, thể hiện khả năng thiết kế giao thức truyền thông tùy chỉnh, không phụ thuộc hoàn toàn vào phần cứng.

1.2 Tóm tắt các nghiên cứu liên quan đã có và kinh nghiệm rút ra

Các nghiên cứu khoa học hiện nay (2019-2025) về đo khoảng cách bằng cảm biến siêu âm cho thấy rõ:

- Ưu điểm: giá thành thấp, dễ triển khai, phù hợp cho nhiều ứng dụng thực tế, phù hợp với đại đa số người dùng cơ bản.
- Nhược điểm: Giao tiếp Trigger/Echo dễ gây sai số, dữ liệu đo chịu ảnh hưởng bởi nhiệt độ, vật liệu hấp thụ âm, góc phản xạ
- Giải pháp hiển thị: thường dùng Serial Monitor hoặc Python GUI đôi khi là mobile APP, nhưng thiếu tính trực quan, khó sử dụng và chưa tự xây dựng chương trình Uart mềm

Từ đó chúng em rút ra kinh nghiệm cho đề tài của nhóm:

- Nên sử dụng chế độ UART mặc định để giảm thiểu tối đa sai số và đơn giản hóa xử lý trong việc xử lý dữ liệu cảm biến gửi về vi xử lý.
- Cần có phần mềm hiển thị trực quan, dễ thao tác và phân tích dữ liệu. Dễ tiếp cận với người dùng phổ thông

- Tự xây dựng UART mềm trên ESP32-S3 để truyền dữ liệu sang WinForm C# sẽ giúp chúng em chủ động định nghĩa khung dữ liệu, tăng tính linh hoạt và phù hợp với yêu cầu giáo viên hướng dẫn đề ra sau đó viết thêm chức năng Test để kiểm tra tốc độ truyền và đánh giá sai số.

1.3 Những khó khăn và vấn đề chưa được giải quyết

Mặc dù đã có nhiều nghiên cứu tuy nhiên vẫn tồn tại một số vấn đề:

- Độ chính xác và ổn định: dữ liệu đo dễ bị nhiễu bởi môi trường, nhiệt độ và bề mặt phản xạ.
- Giao tiếp dữ liệu: các hệ thống trước thường chỉ dùng UART phần cứng, thiếu tính linh hoạt và chưa có giải pháp tùy chỉnh.
- Hiển thị : nhiều hệ thống chỉ hiển thị dữ liệu cơ bản, chưa chính xác, đôi khi bị treo
- Khả năng mở rộng: chưa nhiều hệ thống tích hợp với IoT hoặc có thể truyền dữ liệu qua mạng.

Đề tài này sẽ tập trung giải quyết các vấn đề trên bằng cách:

- Kết hợp US-100 với ESP32-S3 qua UART mặc định để đảm bảo dữ liệu ổn định.
- Xây dựng UART mềm để truyền dữ liệu sang WinForm C#, tạo giao thức truyền thông tùy chỉnh.
- Phát triển phần mềm WinForm C# hiển thị trực quan, phù hợp với đại đa số người dùng phổ thông
- Nghiên cứu cho việc thương mại hóa.

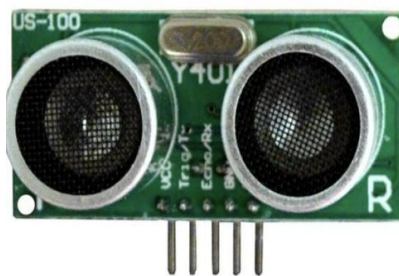
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về các linh kiện chủ đạo

2.1 Cảm biến US100

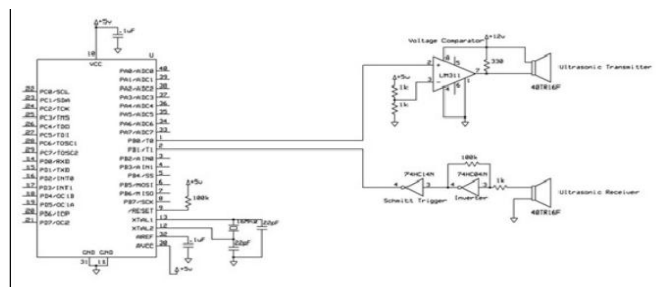
2.1.1 Giới thiệu

Theo tài liệu của Robu.in [1], Mô-đun cảm biến siêu âm US100 là giải pháp tiện lợi để đo khoảng cách đến các vật thể. Mô-đun này có rất nhiều ứng dụng cụ thể như hệ thống giám sát mực nước, xe tránh vật cản, đo khoảng cách trong công nghiệp, v.v. US100 có hiệu suất ổn định và độ chính xác tương đối cao trong khoảng từ 2 cm đến 450 cm.



Hình 2.1.1 : Module US100

Cấu tạo: US-100 gồm 2 bộ phận cấu thành: bộ phận phát và bộ phận thu sóng siêu âm. Mô-đun siêu âm sẽ phát ra một chuỗi gồm tám xung sóng vuông ở tần số 40 kHz từ bộ phát. Khi gặp vật cản, tín hiệu phản hồi sẽ được bộ thu tiếp nhận và chuyển thành dạng sóng có chu kỳ tỷ lệ với khoảng cách đo được. Vi điều khiển kết nối với mô-đun sẽ nhận tín hiệu này và tiến hành xử lý để xác định giá trị khoảng cách. Hình ảnh dưới đây mô tả sơ đồ nguyên lý US100:



Hình 2.1.2 Sơ đồ nguyên lý US100

2.1.2 Nguyên Lý Hoạt Động

Trigger/Echo:

- Vi điều khiển gửi xung Trigger để kích hoạt cảm biến.

- Cảm biến phát sóng siêu âm, nhận tín hiệu phản hồi và xuất ra xung Echo.
- Độ rộng xung Echo tỷ lệ thuận với thời gian truyền/nhận, từ đó tính được khoảng cách với với $v \approx 343 \text{ m/s}$. Hình dưới đây là công thức tính khoảng cách của Module US100:

$$\text{Khoảng cách} = \frac{\text{Thời gian Echo} \times v}{2}$$

2.1.3 Chế độ UART của cảm biến US-100:

Nguyên lý hoạt động:

- Vi điều khiển chỉ cần gửi một lệnh 0x55 đến cảm biến qua giao tiếp UART.
- Sau khi thực hiện phép đo, cảm biến sẽ tự động trả về: 2 byte dữ liệu
 - + ByteHigh (byte cao)
 - + ByteLow (byte thấp)

Khoảng cách được tính theo công thức:

$$\text{Khoảng cách(mm)} = \text{ByteHigh} * 256 + \text{ByteLow}$$

Thông số truyền dữ liệu:

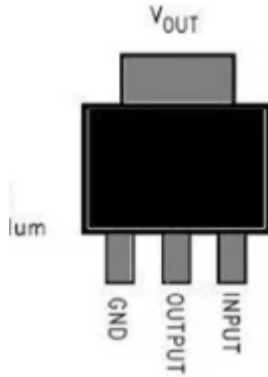
- Tốc độ truyền mặc định: 9600 baud
- Định dạng khung truyền: 8N1 (8 bit dữ liệu, không parity, 1 stop bit)

2.1.4 Các thông số kỹ thuật chính

- Điện áp hoạt động: 5V DC
- Dòng tiêu thụ tĩnh: < 2 mA
- Khoảng cách đo: 2 cm – 450 cm
- Độ chính xác: $\pm 1 \text{ mm}$
- Góc phát hiện: $\leq 15^\circ$
- Tốc độ truyền UART: 9600 baud, 8N1

2.2 SMD LM117-3.3V

Theo datasheet LM117 [2], IC này là bộ ổn áp tuyến tính ba chân (3-terminal adjustable regulator), có khả năng điều chỉnh điện áp đầu ra trong khoảng từ 1.25V đến 37V. Khi cấu hình với các điện trở ngoài phù hợp, LM117 có thể tạo ra mức điện áp ổn định 3.3V, thường dùng cho các mạch số và vi điều khiển

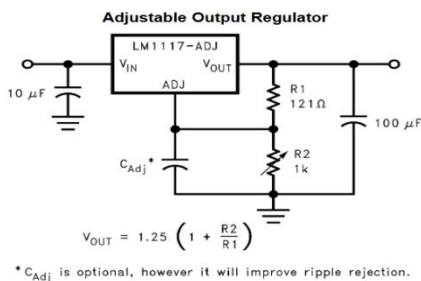


Hình 2.2: Hình dạng IC SMD LM117

2.2.1 Nguyên lý hoạt động

- LM117-ADJ hoạt động bằng cách duy trì điện áp tham chiếu nội bộ $V_{REF} \approx 1.25V$ giữa chân VOUT và chân ADJ.
- Người dùng có thể điều chỉnh điện áp đầu ra bằng cách mắc hai điện trở R1 và R2 tạo thành cầu phân áp giữa VOUT và GND, với điểm nối giữa R1–R2 được đưa về chân ADJ.
- Công thức tính điện áp đầu ra:

$$V_{OUT} = 1.25 \times \left(1 + \frac{R2}{R1}\right) + I_{adj} \times R2$$



Hình 2.2.1: Sơ đồ nguyên lý và công thức tính Vout

Trong đó:

- $V_{REF} \approx 1.25V$
- I_{adj} là dòng điện rất nhỏ (vài μA), thường bỏ qua trong tính toán thực tế.
- Công thức rút gọn thường dùng:

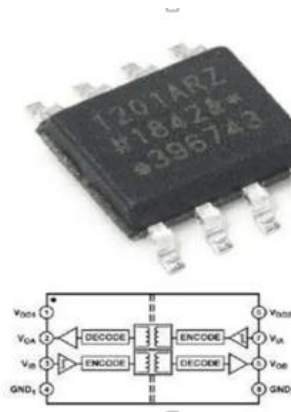
$$V_{OUT} = 1.25 \times (1 + R_2 \div R_1)$$

2.2.2 Thông số kỹ thuật chính

- Điện áp đầu vào (V_{in}): 4.75V – 15V
- Điện áp đầu ra (V_{out}): 1.25V – 13.8V (điều chỉnh được)
- Dòng tải tối đa: 800 mA
- Điện áp rơi (Drop)

2.3 Linh kiện dán SMD ADuM1201ARZ – Bộ cách ly số hai kênh

Theo Analog Devices [3], ADuM1201ARZ là một IC cách ly số hai kênh do hãng Analog Devices sản xuất. IC này sử dụng công nghệ iCoupler® dựa trên biến áp vi mô tích hợp, thay thế cho optocoupler truyền thống. Nhờ đó, ADuM1201ARZ đạt tốc độ truyền dữ liệu cao, độ tin cậy lớn và tiêu thụ công suất thấp.



Hình 2.3.1: Hình dạng và sơ đồ nguyên lý của ADuM1201ARZ

2.3.2. Nguyên lý hoạt động

Nguyên lý hoạt động:

- IC có hai kênh cách ly độc lập, cho phép truyền tín hiệu số qua lớp cách ly điện mà không cần kết nối trực tiếp.
- Công nghệ iCoupler® có nhiệm vụ biến áp vi mô để truyền tín hiệu, đảm bảo cách ly điện áp lên đến 2.5 kVrms trong 1 phút.
- Hỗ trợ truyền dữ liệu **một chiều** (unidirectional) với tốc độ lên đến 25 Mbps.
- Độ méo xung (pulse width distortion) rất thấp, khoảng 3 ns, giúp tín hiệu chính xác.
- ADuM1201ARZ được sử dụng để cách ly tín hiệu UART mềm giữa vi điều khiển và ứng dụng WinForms C#. Việc cách ly giúp bảo vệ máy tính khỏi nhiễu và sự cố điện áp từ mạch công suất, đồng thời đảm bảo tín hiệu truyền thông ổn định.

2.3.3 Thông số kỹ thuật chính

- Điện áp hoạt động: 2.7V – 5.5V
- Số kênh cách ly: 2 kênh (1 kênh vào, 1 kênh ra)
- Tốc độ truyền dữ liệu: DC đến 25 Mbps
- Dòng tiêu thụ:
 - o 1.1 mA/kênh (0–2 Mbps, 5V)
 - o 3.7 mA/kênh (10 Mbps, 5V)
 - o 8.2 mA/kênh (25 Mbps, 5V)
- Điện áp cách ly: 2500 Vrms trong 1 phút
- Nhiệt độ hoạt động: –40°C đến +125°C

2.4 Vi xử lý ESP32-S3 WROOM-1

2.4.1 Giới thiệu

Theo Espressif Systems[4], ESP32-S3 WROOM-1 là một vi điều khiển do hãng Espressif Systems sản xuất. Module này tích hợp vi xử lý Xtensa® dual-core 32-bit LX7 với tốc độ lên đến 240 MHz, cùng bộ nhớ Flash 4 MB(phiên bản MCN4), phù hợp cho các ứng dụng IoT, AI và thiết bị nhúng cần hiệu năng vừa phải nhưng vẫn đảm bảo chi phí thấp.

- Được tích hợp dao động nội (oscillator), giúp hệ thống có thể chạy độc lập mà không cần thạch anh ngoài.
- Bộ nhớ *Flash 4 MB* được tích hợp sẵn để lưu chương trình và dữ liệu.
- Bộ nhớ SRAM/DRAM dung lượng lớn cho phép xử lý các ứng dụng phức tạp, hỗ trợ AI acceleration instructions để tăng tốc các tác vụ machine learning như nhận dạng giọng nói hoặc hình ảnh.
- Có 36 GPIO với nhiều giao tiếp: UART, SPI, I2C, PWM, ADC, DAC...
- Cho phép lập trình bằng Arduino IDE, ESP-IDF hoặc MicroPython, thuận tiện cho phát triển ứng dụng.

2.4.3 Thông số kỹ thuật chính

- CPU: Xtensa® dual-core LX7, 240 MHz
- Bộ nhớ:
 - 384 KB ROM
 - 512 KB SRAM
 - 16 KB SRAM trong RTC
 - Flash: 4 MB (phiên bản MCN4)
 - PSRAM: tùy chọn (0 MB)
- Dao động nội: tích hợp sẵn, không cần thạch anh ngoài
- Có thể Kết nối Wi-Fi 2.4 GHz (802.11 b/g/n), Bluetooth 5 LE nếu cần
- GPIO: 36 chân, hỗ trợ nhiều giao tiếp ngoại vi
- Điện áp hoạt động: 3.0V – 3.6V
- Dòng tiêu thụ:
 - ~240 mA khi Wi-Fi TX ở công suất cao
 - ~80 mA khi CPU chạy ở chế độ thường
 - ~20 μ A ở chế độ deep sleep
- Công suất phát Wi-Fi: tối đa +20 dBm

- Nhiệt độ hoạt động: -40°C đến $+85^{\circ}\text{C}$
- Đóng gói: module SMD, dễ hàn trên PCB

2.4.4 Ứng dụng thực tế trong đề tài

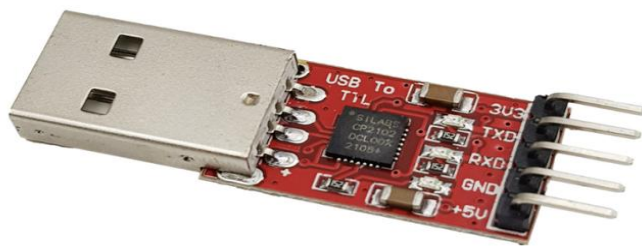
Trong đề tài này, ESP32-S3 WROOM-1 được sử dụng làm *vi xử lý trung tâm*, đảm nhiệm:

- Thu thập dữ liệu từ cảm biến (ví dụ US-100).
- Giao tiếp UART với ADuM1201 để cách ly tín hiệu khi truyền về PC.
- Kết nối Wi-Fi/Bluetooth để truyền dữ liệu đến ứng dụng WinForms C#.
- Tận dụng dao động nội và bộ nhớ Flash/SRAM tích hợp để giảm linh kiện ngoại vi, tối ưu thiết kế PCB.

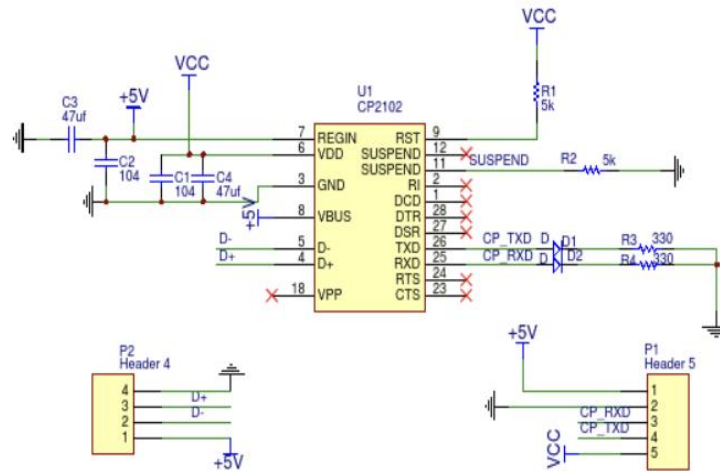
2.5 CP2102 Module (USB to TTL)

2.5.1 Giới thiệu

CP2102 là một vi mạch chuyển đổi giao tiếp từ USB sang UART, được thiết kế nhằm đơn giản hóa việc kết nối giữa máy tính và các thiết bị sử dụng giao tiếp nối tiếp. Với khả năng tích hợp cao, CP2102 giúp loại bỏ nhu cầu sử dụng các linh kiện phụ trợ như thạch anh ngoài, từ đó tiết kiệm không gian và chi phí trong thiết kế phần cứng. Trong đề tài này chúng em sử dụng Module CP2102 để thực hiện 2 chức năng chính đó là nạp code và giao tiếp với Winform C#. Nhờ tính ổn định, dễ sử dụng và khả năng tương thích cao, CP2102 đã trở thành một lựa chọn của nhóm chúng em để hỗ trợ đề tài trong môn học này.



Hình 2.5.1: CP2102 Module (USB to TTL)



Hình 2.5.2: Sơ đồ nguyên lý của CP2102 Module (USB to TTL)

2.5.2 Nguyên lý hoạt động

CP2102 là một vi mạch chuyển đổi giao tiếp từ USB sang UART, được thiết kế nhằm đơn giản hóa quá trình kết nối giữa máy tính và các thiết bị sử dụng giao tiếp nối tiếp. Nhờ khả năng tích hợp cao, CP2102 loại bỏ nhu cầu sử dụng các linh kiện phụ trợ như thạch anh ngoài hay mạch điều khiển riêng biệt, từ đó giúp tối ưu hóa không gian và chi phí trong thiết kế phần cứng. Khi được kết nối, CP2102 đóng vai trò như một cầu nối giữa cổng USB của máy tính và cổng UART của các thiết bị như vi điều khiển, module cảm biến hoặc hệ thống nhúng. Vi mạch này tạo ra một cổng COM ảo trên máy tính, cho phép phần mềm giao tiếp với thiết bị thông qua giao thức nối tiếp một cách thuận tiện và hiệu quả.

2.5.3 Thông số kĩ thuật chính

- Phạm vi nhiệt độ: -40Cto + 85C
- Hỗ trợ windows vista / xp / server 2003/200, Mac OS-X / OS-9, Linux
- Màu chính: đỏ
- Kích thước: chiều dài (Không bao gồm USB): 30 mm
- USB để lấy nguồn, dẫn đến giao diện bao gồm 3,3V (<40mA), 5V, GND, TX, RX, mức pin tín hiệu là 3,3V, logic dương
- Hỗ trợ tốc độ truyền trong khoảng 300bps ~ 1Mbps

2.6 Tổng quan về phần mềm, giao thức Uart, phương pháp đo và thuật toán

2.6.1 Phần mềm WinForms C#

WinForms (Windows Forms) là một thư viện giao diện đồ họa (GUI) của Microsoft, được tích hợp trong .NET Framework và .NET Core. Đây là công cụ phổ biến để xây dựng ứng dụng desktop trên Windows. Ở đề tài này chúng em sẽ sử dụng ngôn ngữ C# để thực hiện tạo ra 1 giao diện điều khiển đơn giản, trực quan và dễ sử dụng.

- Đặc điểm chính:
 - o Thiết kế giao diện bằng cách kéo thả (drag-and-drop).
 - o Hỗ trợ nhiều control: Button, TextBox, Label, DataGridView...
 - o Tích hợp dễ dàng với thư viện .NET để xử lý dữ liệu, giao tiếp ngoại vi.
 - o Có thể kết hợp với SerialPort class để giao tiếp UART/COM với vi điều khiển.
- Ứng dụng trong đề tài:
 - o Xây dựng phần mềm giao diện trên PC để hiển thị dữ liệu từ vi điều khiển.
 - o Cho phép người dùng nhập lệnh, cấu hình thông số và nhận dữ liệu phản hồi.
 - o Giao tiếp với ESP32-S3 thông qua UART mềm đã được cách ly bằng ADuM1201.

2.6.2 Phần mềm lập trình Firmware (VS Code + PlatformIO)

Firmware là chương trình nhúng chạy trực tiếp trên vi điều khiển (cụ thể như ESP32-S3). Để lập trình firmware, ta sử dụng Visual Studio Code (VS Code) kết hợp với PlatformIO IDE.

- Visual Studio Code (VS Code):
 - o Trình soạn thảo mã nguồn đa nền tảng, nhẹ, mạnh mẽ.
 - o Hỗ trợ nhiều ngôn ngữ lập trình (C, C++, Python...).
 - o Có hệ thống extension phong phú, trong đó có PlatformIO.
- PlatformIO IDE:

- Extension của VS Code, chuyên cho phát triển nhúng.
- Hỗ trợ nhiều board và vi điều khiển: ESP32, STM32, Arduino...
- Tích hợp sẵn trình biên dịch, upload firmware, debug.
- Quản lý thư viện dễ dàng, có cộng đồng lớn.
- Cho phép viết code C/C++ cho ESP32-S3, biên dịch và nạp trực tiếp qua cổng USB.
- Ứng dụng trong đề tài:
 - Lập trình firmware cho ESP32-S3 để thu thập dữ liệu từ cảm biến US-100.
 - Giao tiếp UART với ADuM1201 để truyền dữ liệu về PC.
 - Kết nối Wi-Fi/Bluetooth để mở rộng khả năng truyền dữ liệu.
 - Tận dụng thư viện của PlatformIO để rút ngắn thời gian phát triển.

2.6.3 Cơ sở lý thuyết về thuật toán và phương pháp xử lý dữ liệu

Trong hệ thống vi điều khiển, việc truyền nhận và xử lý dữ liệu cần dựa trên một số nguyên lý cơ bản để đảm bảo tính chính xác và ổn định:

- Nguyên lý truyền thông nối tiếp (UART): UART là chuẩn giao tiếp phổ biến giữa vi điều khiển và thiết bị ngoại vi. Có thể sử dụng cả UART phần cứng và UART mô phỏng bằng phần mềm (Soft UART) để thực hiện việc gửi và nhận dữ liệu.
- Cấu trúc gói tin: Dữ liệu thường được đóng gói theo một định dạng nhất định, có ký hiệu bắt đầu và kết thúc để dễ dàng nhận diện. Điều này giúp hệ thống phân biệt rõ ràng từng khung dữ liệu trong quá trình truyền.
- Thuật toán kiểm tra lỗi (CRC): Để đảm bảo dữ liệu không bị sai lệch trong quá trình truyền, hệ thống sử dụng thuật toán kiểm tra lỗi như CRC. Giá trị CRC được tính từ dữ liệu gốc và gửi kèm theo gói tin, giúp bên nhận xác minh tính toàn vẹn của dữ liệu.
- Xử lý dữ liệu cảm biến: Các cảm biến, chẳng hạn cảm biến siêu âm US100, sẽ gửi thông tin đo đạc về vi điều khiển thông qua giao tiếp UART. Vi điều khiển tiếp nhận các giá trị này, thực hiện bước xử lý cần thiết và truyền kết quả đến hệ thống giám sát hoặc phần mềm giao diện để phục vụ việc theo dõi và phân tích.
- Chu trình hoạt động: Hệ thống thực hiện song song hai nhiệm vụ:
 - Nhận và phản hồi dữ liệu từ phần mềm giao diện.

- Đọc dữ liệu từ cảm biến theo chu kỳ, sau đó truyền đi dưới dạng gói tin có kiểm tra lỗi.

2.6.4 Thuật toán kiểm tra lỗi (CRC)

CRC là gì ?

CRC (Cyclic Redundancy Check) là một cách để phát hiện lỗi khi dữ liệu được truyền đi hoặc lưu trữ. Bạn có thể hình dung nó giống như việc thêm một “mã bảo vệ” vào cuối dữ liệu để người nhận kiểm tra xem dữ liệu có bị sai sót trong quá trình truyền không.

Nguyên lý hoạt động:

- Khi gửi dữ liệu, hệ thống sẽ tính toán một con số đặc biệt (gọi là CRC) dựa trên nội dung dữ liệu.
- Con số này được gắn kèm theo dữ liệu gửi đi.
- Bên nhận cũng làm lại phép tính đó.
- Nếu kết quả giống nhau → dữ liệu ok. Nếu khác → dữ liệu đã bị lỗi.

Ưu điểm:

- Phát hiện được nhiều loại lỗi: sai một bit, sai nhiều bit, hoặc lỗi liên tiếp.
- Tính toán nhanh, dễ làm bằng phần cứng.
- Được dùng rộng rãi trong lĩnh vực IOT..

Ví dụ ứng dụng CRC trong hệ thống US-100 → ESP32-S3 → WinForms C#:

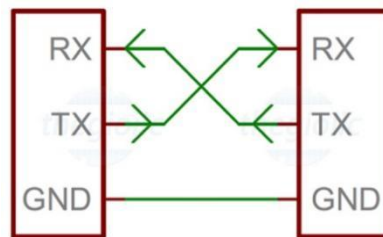
Giả sử ta có một hệ thống đo khoảng cách bằng cảm biến siêu âm US-100. Khi cảm biến đo được khoảng cách, nó sẽ gửi dữ liệu dạng số (ví dụ: 101100 tương ứng với khoảng cách 44 cm) về vi điều khiển ESP32-S3. Trước khi ESP32-S3 truyền dữ liệu này lên máy nó sẽ tính toán thêm một mã CRC để đảm bảo dữ liệu không bị sai trong quá trình truyền. Cụ thể:

- ESP32-S3 nhận dữ liệu từ US-100
 - Ví dụ dữ liệu đo được là 101100.
- Tính CRC trên ESP32-S3
 - ESP32-S3 chọn một đa thức sinh (ví dụ 1101).
 - Thực hiện phép chia nhị phân, thu được phần dư CRC (ví dụ 011).

- Ghép dữ liệu và CRC thành 101100011.
- Gửi dữ liệu lên WinForms C#
 - ESP32-S3 truyền chuỗi 101100011 qua CP2102
 - Ứng dụng WinForms nhận chuỗi này.
- WinForms kiểm tra CRC
 - Chương trình C# thực hiện lại phép chia với đa thức sinh 1101.
 - Nếu dư bằng 000 → dữ liệu hợp lệ, hiển thị khoảng cách đo được.
 - Nếu dư khác 000 → báo lỗi, dữ liệu bị sai.

2.6.5 Giao thức UART

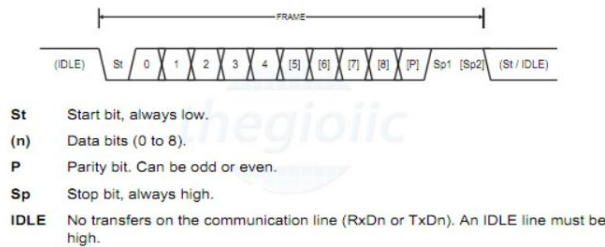
UART (Universal Asynchronous Receiver/Transmitter) là giao thức truyền thông nối tiếp bất đồng bộ, rất phổ biến trong các hệ thống nhúng.



Hình 2.3.II: mô tả giao thức Uart

- Nguyên lý hoạt động:
 - UART truyền dữ liệu theo dạng khung (frame) gồm: bit start, dữ liệu (8 hoặc 9 bit), bit parity (tùy chọn), bit stop. Phổ biến là khung truyền 8N1
 - Tốc độ truyền được xác định bằng baud rate (ví dụ: 9600, 115200 bps).

- Gồm hai đường tín hiệu chính: TX (Transmit) và RX (Receive). Nguyên lý hoạt động được mô tả khái quát qua hình bên dưới:



- Vai trò trong đề tài:
 - ESP32-S3 gửi dữ liệu cảm biến qua UART.
 - ADuM1201 cách ly tín hiệu UART để bảo vệ PC.
 - WinForms C# nhận dữ liệu qua UART mềm và hiển thị trên giao diện.
- Ưu điểm:
 - Đơn giản, dễ triển khai.
 - Phổ biến, hầu hết vi điều khiển và PC đều hỗ trợ.
 - Tốc độ đủ cho các ứng dụng truyền dữ liệu cảm biến.

2.6.6 UART cứng là gì

UART cứng là bộ mạch phần cứng tích hợp sẵn trong vi xử lý để thực hiện giao tiếp nối tiếp.

Nó cho phép truyền và nhận dữ liệu theo chuẩn UART một cách tự động, không cần CPU phải canh thời gian từng bit. Ưu điểm của UART cứng là tốc độ cao, độ tin cậy tốt và giảm tải cho CPU, nhưng số lượng cổng UART thường bị giới hạn theo thiết kế của chip.

2.6.7 UART mềm là gì

UART mềm là phương pháp mô phỏng giao tiếp UART bằng phần mềm trên vi xử lý. Thay vì sử dụng bộ UART phần cứng tích hợp sẵn, người lập trình chọn hai chân GPIO bất kỳ để làm TX/RX, và điều khiển chúng theo đúng chuẩn truyền nối tiếp (start bit, data bit, stop bit). Cách này cho phép mở rộng số lượng cổng UART, nhưng đòi hỏi CPU phải xử lý chính xác về thời gian và thường tiêu tốn nhiều tài nguyên hơn so với UART phần cứng. Song song đó tín hiệu Uart mềm dễ bị nhiễu vì thế trong đề tài này nhóm em đã sử dụng thêm một linh kiện AduM1201ARZ để cách ly giúp bảo vệ máy tính khỏi nhiễu.

2.6.8 Bản so sánh UART cứng và UART mềm

Tiêu chí	UART cứng	UART mềm
Cách thực hiện	Dùng mạch phần cứng tích hợp sẵn trong vi xử lý	Dùng phần mềm điều khiển GPIO để giả lập UART
Độ chính xác thời gian	Rất cao, do phần cứng chuyên dụng đảm bảo	Phụ thuộc vào CPU và lập trình, dễ sai lệch
Tải cho CPU	Thấp, CPU chỉ đọc/ghi thanh ghi	Cao, CPU phải canh thời gian từng bit
Tốc độ truyền	Nhanh và ổn định	Thường chậm hơn, dễ bị nhiễu
Số lượng cổng	Bị giới hạn theo thiết kế của chip	Có thể mở rộng bằng cách chọn thêm GPIO
Ứng dụng	Giao tiếp chuẩn, yêu cầu độ tin cậy cao	Khi thiếu UART cứng hoặc cần thử nghiệm linh hoạt

Bản so sánh chi tiết UART cứng và UART MỀM

2.6.9 Phương pháp đo điện áp và dòng tiêu thụ

Khi kiểm tra độ ổn định và hiệu quả hoạt động của hệ thống điện tử, việc đo công suất tiêu thụ là bước không thể bỏ qua.

- Kiểm tra điện áp: Dùng đồng hồ vạn năng ở chế độ đo điện áp một chiều. Đặt que đo song song với các đường cấp nguồn, chẳng hạn VDD–GND của vi điều khiển ESP32 S3 hoặc VCC–GND của cảm biến US100. Mục đích là xác định điện áp thực tế có đúng với thiết kế, xem có hiện tượng sụt áp không và có duy trì ổn định trong quá trình hoạt động hay không.
- Kiểm tra dòng điện tiêu thụ: Sử dụng bộ nguồn đa năng chúng em có thể đo dòng tiêu thụ một cách rất dễ dàng. Sau khi cấp nguồn cho bộ nguồn chúng em thiết lập mức điện áp cấp vào Board mạch là 5V, sau đó tiến hành cấp vào Board mạch của nhóm và lập tức trên khối nguồn sẽ hiện ra dòng điện tiêu thụ.

2.6.10 Phương pháp đo và quan đánh giá tín hiệu Wavefrom bằng máy hiện sóng

Sau khi cấp nguồn cho máy hiện sóng nhóm em tiến hành chọn kênh một hoặc hên hai để chuẩn bị đo dạng tín hiệu Uart mềm và Uart Us100. Đầu tiên nhóm tiến hành kẹp chân TX sau đó kẹp chân GND và chỉnh ở chế độ xung vuông sau khi cân chỉnh (trục, độ rộng, chu kì...) nhóm tiến hành quan sát trên màn hình hình dạng xung, thời gian T-on, T-off, tốc độ truyền có đúng 9600 hay không, dây điện áp hoạt động và đánh giá xung nhiễu. Đối với Uart mềm chúng em sẽ tiến hàng viết một chương trình gửi chữ “A” 1000 lần và sau đó tiến hành đo tín hiệu ở máy hiện sóng, song song đó tiến hành phân tích dạng sóng để kiểm chứng.

2.6.11 Phương pháp đánh giá phần mềm

Đầu tiên vi xử lý sẽ đọc giá trị cảm biến US100, Uart mềm sẽ sử dụng tốc độ 9600, Winform C# sẽ gửi 1000 frame xuống vi xử lý, vi xử lý nhận được 1000 frame sau đó gửi ngược lại Winform C#. Nếu cả hai quá trình diễn ra trơn tru thì phần mềm sẽ hiển thị bản “ Kết quả Test”, tại bản này sẽ có các thông số cơ bản như sau:

- Gửi bao nhiêu
- Nhận bao nhiêu
- Dung bao nhiêu
- Tỷ lệ đúng
- Thời gian và tốc độ

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

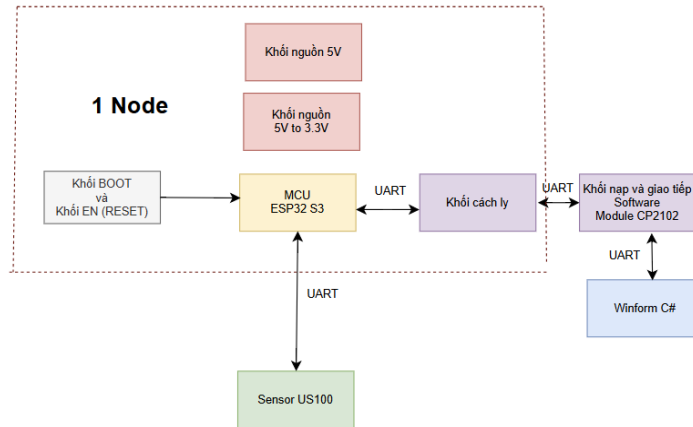
3.1 Phân tích thiết kế hệ thống

3.1.1 Phân tích, xác định thông số đầu vào đầu ra của hệ thống

- Vi xử lý trong tâm ESP32-S3 WROOM-1 (phiên bản MCN4) đã tích hợp bộ nhớ Flash, và giao động nội
- Cảm biến khoảng cách US100 Đo khoảng cách và truyền tín hiệu (Uart) về vi xử lý. Có thể thiết kế mạch lọc RC nếu tín hiệu bị nhiễu
- Thiết kế khối nguồn: hạ áp từ 5V xuống 3.3V cấp cho vi xử lý và các IC khác
- Thiết kế khối cách ly có khả năng cách ly tín hiệu Uart từ vi xử lý đến Winform C# và ngược lại
- Thiết kế khối nạp code bằng Uart mặc định, và sử dụng 2 chân uart mặc định của vi xử lý để cấu hình Uart mềm trong code Firmware và cả 2 Uart (mặc định và mềm) đều được cách ly bằng AduM1201ARZ sau đó qua module CP2102 để nạp code (sử dụng Uart mặc định) hoặc giao tiếp Winform C# (Uart mềm)
- Cần sử dụng Uart mặc định của vi xử lý để đọc dữ liệu cảm biến US100, sau đó thiết lập chân Uart mềm để giao tiếp với Winform C#
- Điều khiển thông qua Winform C#, có các chức năng cơ bản đồng thời phải có chức năng kiểm tra độ ổn định của Uart mềm

3.1.2 Thiết kế sơ đồ khối

Thiết kế sơ đồ khối: Để thiết kế và triển khai hệ thống đo khoảng cách bằng cảm biến siêu âm một cách khoa học và hiệu quả, việc xây dựng sơ đồ khối tổng quát là bước đầu tiên và quan trọng. Sơ đồ khối giúp hình dung rõ ràng cấu trúc toàn hệ thống, thể hiện mối liên kết giữa các thành phần chính, từ khâu thu nhận tín hiệu khoảng cách bằng cảm biến US-100, xử lý dữ liệu trên vi điều khiển ESP32-S3, cho đến truyền thông qua giao tiếp UART và hiển thị kết quả trên phần mềm WinForms C#. Mỗi khối chức năng trong sơ đồ đảm nhận một nhiệm vụ riêng biệt nhưng phối hợp nhịp nhàng, nhằm đảm bảo hệ thống hoạt động chính xác, ổn định và dễ dàng mở rộng. Dưới đây là sơ đồ khối tổng quát của hệ thống đo khoảng cách mà nhóm đã thực hiện, thể hiện trong hình 3.1.2.



Hình 3.1.2: sơ đồ khối của hệ thống

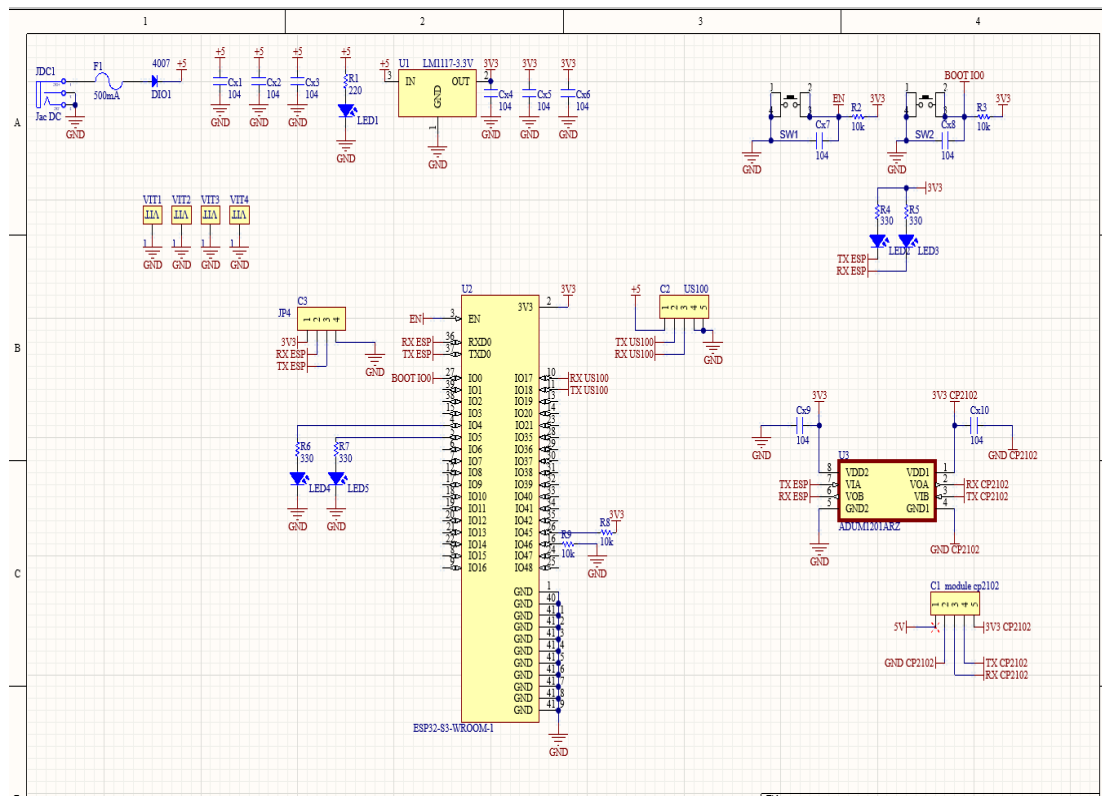
Chức năng các khối: Sơ đồ khối của mạch đồng hồ số được thiết kế nhằm thể hiện rõ cấu trúc chức năng và nguyên lý hoạt động của hệ thống. Mỗi thành phần trong sơ đồ đảm nhiệm một vai trò cụ thể, được phân chia theo từng nhiệm vụ cụ thể như sau:

- Khởi nguồn: có nhiệm vụ cung cấp điện áp ổn định cho vi xử lý cũng như các linh kiện cần nguồn cấp. Việc tách biệt nguồn theo từng phần giúp giảm nhiễu và đảm bảo sự ổn định trong quá trình hoạt động.
- Khởi EN(enable): đảm bảo hệ thống có thể reset nhanh khi cần[4].
- Khởi BOOT: cho phép nạp lại firmware cho ESP32-S3 khi phát triển hoặc cập nhật chương trình. Khi hệ thống chạy thực tế, BOOT thường để mặc định (HIGH) để chip tự chạy chương trình đã nạp[4].
- Khởi cách ly AduM1201: Trong hệ thống có nhiệm vụ đảm bảo an toàn điện và độ ổn định khi truyền dữ liệu UART giữa vi điều khiển ESP32-S3 và phần mềm WinForms C# trên máy tính. Tuy nhiên chúng em đã thiết kế tối ưu ở chỗ “*chỉ 2 chân nhưng nó mang lại tận 2 công năng là nạp code và giao tiếp với Winform C#, nghĩa là sử dụng duy nhất 2 chân uart mặc định của vi xử lý để cấu hình Uart mềm trong code Firmware song song đó cả 2 Uart (mặc định và mềm) đều được cách ly bằng AduM1201ARZ sau đó qua module CP2102 để nạp code (sử dụng Uart mặc định) hoặc giao tiếp Winform C# (Uart mềm)*”. Trong đề tài, ESP32 sử dụng UART mềm để gửi dữ liệu đo khoảng cách, và khối ADuM1201 được đặt giữa hai chân UART mềm và module chuyển đổi CP2102 nhằm cách ly hoàn toàn về mặt điện giữa vi điều khiển và máy tính và có thể nạp code thông

qua Uart mặc định. Nhờ công nghệ iCoupler tích hợp trong ADuM1201, tín hiệu số được truyền qua vùng cách ly mà không cần kết nối điện trực tiếp, giúp ngăn chặn nhiễu, xung điện và sai lệch mức áp từ phía máy tính, đồng thời bảo vệ vi điều khiển khỏi các sự cố phần cứng tiềm ẩn [10].

3.1.3 Thiết kế nguyên lý

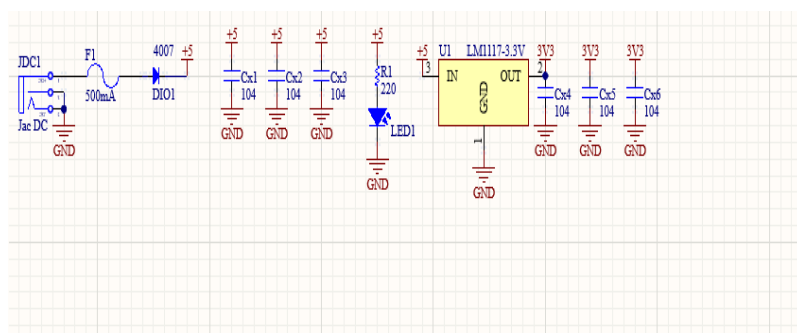
Sơ đồ nguyên lý của dự án được trình bày ở hình 3.1.3, thể hiện đầy đủ các khối chức năng chính của hệ thống đo khoảng cách bằng cảm biến siêu âm US-100, sử dụng vi điều khiển ESP32-S3 và giao tiếp với phần mềm WinForms C# thông qua module CP2102. Trong hệ thống này, cảm biến US-100 hoạt động ở chế độ UART, gửi dữ liệu khoảng cách dưới dạng khung 2 byte đến ESP32-S3 thông qua giao tiếp UART cứng. Vi điều khiển ESP32-S3 xử lý dữ liệu nhận được, sau đó truyền kết quả khoảng cách đến máy tính bằng giao tiếp UART mềm do người lập trình tự xây dựng. Để đảm bảo an toàn điện và chống nhiễu, tín hiệu UART mềm được cách ly bằng mạch ADuM1201 trước khi đến CP2102. Module CP2102 chuyển đổi tín hiệu UART sang USB, cho phép phần mềm WinForms C# trên máy tính nhận và hiển thị dữ liệu theo thời gian thực. Toàn bộ hệ thống được cấp nguồn ổn định từ mạch nguồn 3.3V, có tích hợp diode bảo vệ và tụ lọc để đảm bảo độ tin cậy khi vận hành.



Hình 3.1.3: sơ đồ nguyên lý

3.1.4 Thiết kế bộ hạ áp 5V to 3.3V

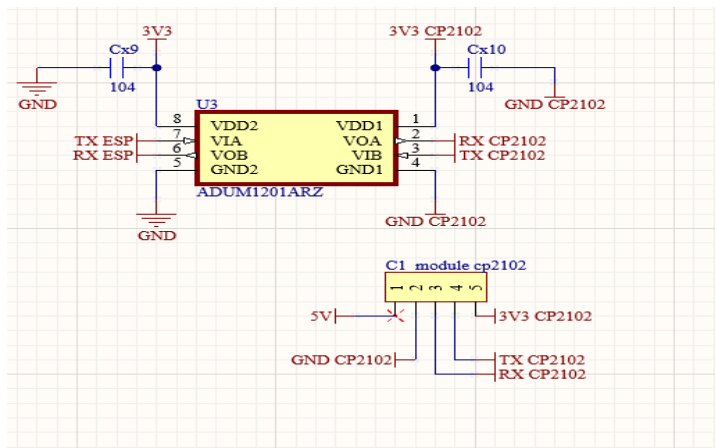
Sơ đồ nguyên lý của bộ chuyển đổi điện áp từ 5V xuống 3.3V được trình bày ở hình 3.1.4. Mạch sử dụng IC ổn áp tuyến tính SMD LM1117-3.3 để tạo ra nguồn 3.3V ổn định từ nguồn đầu vào 5V DC. Đầu vào được cấp thông qua jack DC, đi qua cầu chì bảo vệ dòng (500mA) nhằm ngăn ngừa quá tải. Tiếp theo là diode 1N4007 để bảo vệ phân cực ngược, tránh hư hỏng linh kiện khi cắm sai chiều nguồn tại đây, ta tiến hành lọc nhiễu 5V bằng các tụ gốm 104(0.1 μ F) để có thể giảm nhiễu cao tần một cách tối đa. IC SMD LM1117-3.3 nhận điện áp đầu vào và tiến hành điều chỉnh xuống mức 3.3V tại đầu ra. Tại ngõ ra, các tụ lọc tiếp tục được sử dụng để ổn định điện áp và giảm dao động cũng như để lọc nhiễu đảm bảo tín hiệu sạch để cấp vào vi xử lý. Ngoài ra, mạch còn tích hợp đèn LED báo nguồn, nối qua điện trở hạn dòng 220 Ω , giúp người dùng dễ dàng nhận biết trạng thái hoạt động của mạch. Bộ nguồn này đóng vai trò quan trọng trong hệ thống, cung cấp điện áp phù hợp cho ESP32-S3 và các linh kiện hoạt động ở mức 3.3V, đảm bảo độ tin cậy và an toàn khi vận hành.



Hình 3.1.4.: sơ đồ nguyên lý mạch nguồn

3.1.5 Thiết kế Opto chống nhiễu Uart

Sơ đồ nguyên lý của mạch cách ly tín hiệu UART giữa ESP32-S3 và module CP2102 được trình bày ở hình 3.1.5. Trong thiết kế này, vi điều khiển ESP32-S3 sử dụng giao tiếp UART mềm để truyền dữ liệu đo khoảng cách đến máy tính. Để đảm bảo an toàn điện và tránh nhiễu từ phía máy tính, tín hiệu UART được đưa qua mạch cách ly số sử dụng IC ADuM1201. IC này có hai kênh cách ly độc lập, cho phép truyền tín hiệu TX và RX giữa ESP32 và CP2102 mà không có kết nối điện trực tiếp. Phía ESP32 kết nối với các chân VIA và VOB của ADuM1201, trong khi phía CP2102 kết nối với VOA và VIB. Hai vùng cách ly được cấp nguồn riêng biệt bằng điện áp 3.3V, có tụ lọc để ổn định tín hiệu. Sau khi tín hiệu được cách ly, CP2102 chuyển đổi UART sang USB và truyền đến phần mềm WinForms C# trên máy tính để hiển thị kết quả đo. Song song đó khối này vẫn có thể nạp code cho ESP32 S3 được bởi vì chính bản thân 2 chân TX RX này bản chất nó là hai chân Uart mặc định của vi xử lý. Thiết kế này giúp bảo vệ vi điều khiển khỏi xung điện, sai lệch mức áp hoặc nhiễu từ máy tính, đồng thời đảm bảo giao tiếp UART mềm hoạt động ổn định và chính xác. Từ đó ta nhận thấy chỉ cần một khối nhưng có thể sử dụng ở hai chế độ khác nhau.

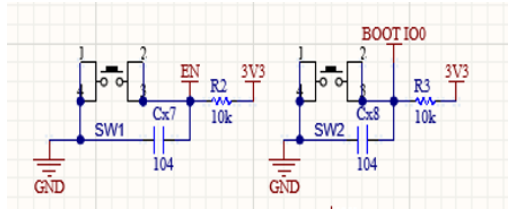


Hình 3.1.5 sơ đồ nguyên lý Opto chống nhiễu

3.1.6 Thiết kế khối Enabl và Boot

Hai sơ đồ nguyên lý (hình 3.1.6) sử dụng hai nút nhấn SW1 (EN) và SW2 (BOOT IO0) để phục vụ quá trình reset và đưa ESP vào chế độ nạp code. Mỗi nút đều dùng nguồn 3V3, điện trở 10kΩ và tụ 100nF (ký hiệu “104”) tạo mạch RC nhằm kéo mức logic ổn định và chống dội.

- 1) Khối reset (SW1 – EN): EN là chân enable của ESP. Điện trở 10kΩ cấu hình kéo lên 3V3 để giữ ESP ở trạng thái hoạt động bình thường. Khi nhấn SW1, EN bị kéo xuống GND làm thiết bị reset. Tụ 100nF tạo trễ ngắn và lọc nhiễu, giúp xung reset sạch, tránh reset giả.
- 2) Khối boot (SW2 – BOOT IO0): IO0 được kéo lên 3V3 qua điện trở 10kΩ để ở mức cao khi bình thường. Khi cần nạp code, giữ SW2 (kéo IO0 xuống GND) đồng thời nhấn reset (SW1) để ESP khởi động vào chế độ bootloader UART. Tụ 100nF lọc và chống dội, đảm bảo mức thấp trên IO0 đủ ổn định trong thời điểm reset để kích hoạt chế độ nạp.
- 3) Cách thao tác nạp code: Nhấn giữ SW2 (BOOT IO0), nhấn nhả SW1 (EN) để reset, sau đó thả SW2. ESP sẽ vào chế độ bootloader và sẵn sàng nhận firmware qua UART mềm và được cách lý sau đó chuyển đổi bằng Module CP2102 để giao tiếp với máy tính.



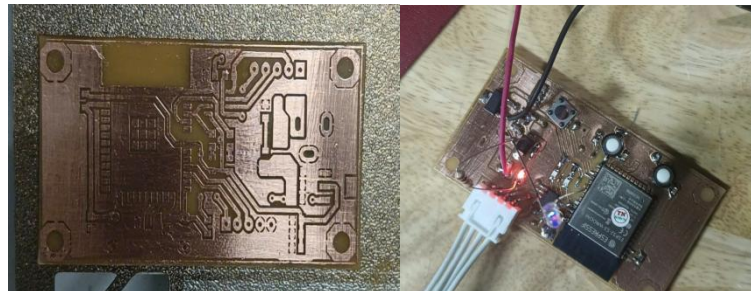
Hình 3.1.6: Sơ đồ nguyên lý khối En và Boot

3.2 Thi công

3.2.1 Vẽ mạch in

Vẽ mạch in là một bước quan trọng trong quá trình thiết kế và chế tạo hệ thống đo khoảng cách. Việc thực hiện đúng quy trình vẽ mạch in giúp đảm bảo tính chính xác, độ ổn định và khả năng sản xuất của mạch. Kết quả mạch in của hệ thống sử dụng ESP32-S3, mạch nguồn, cảm biến US-100, mạch cách ly ADuM1201 và giao tiếp WinForms thông qua CP2102 và nạp code ,các cổng,các linh kiện hỗ trợ khác được thể hiện qua hình 3.2.1.2. Mạch được thiết kế dưới dạng PCB hai lớp, phù hợp với yêu cầu công nghiệp về độ ổn định, khả năng chống nhiễu và tối ưu không gian bố trí linh kiện.

Trước khi tiến hành vẽ mạch in công nghiệp, *tất cả thành viên nhóm* đã thực hiện vẽ mạch in thủ công và test các chức năng phù hợp với yêu cầu đề tài và đã đúc kết được kinh nghiệm cũng như nhận thấy mạch hoạt động tương đối tốt. Minh chứng được thể hiện cụ thể tại hình 3.2



Hình 3.2: Làm PCB thủ công

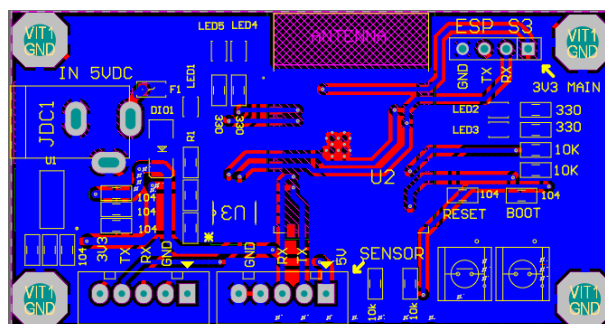
Sau đó tiến hành vẽ mạch in 2 lớp:

- 1) Bước 1: Mở phần mềm Altium Designer với giao diện thiết kế PCB Sau khi hoàn thành sơ đồ nguyên lý, tiến hành chuyển sang chế độ thiết kế mạch in bằng cách vào

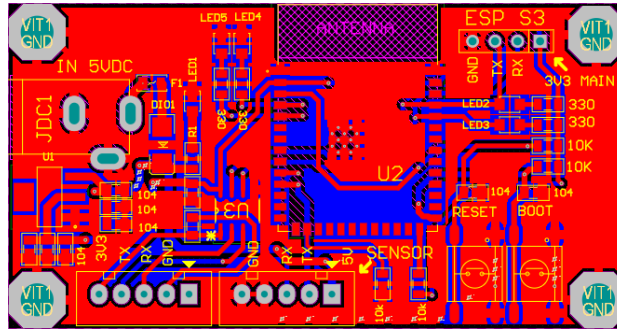
menu "Tools" và chọn "Switch to PCB Layout". Giao diện PCB sẽ hiển thị toàn bộ linh kiện đã khai báo, sẵn sàng cho quá trình bố trí.

- 2) Bước 2: Bố trí linh kiện trên mặt mạch Các linh kiện như ESP32-S3, cảm biến US-100(2 chân RX,TX được thể hiện trong Header) , IC cách ly ADuM1201, module CP2102(2 chân RX,TX được thể hiện trong Header) và các linh kiện phụ trợ sẽ xuất hiện trong vùng làm việc. Tiến hành sắp xếp các linh kiện sao cho hợp lý, đảm bảo đường đi tín hiệu ngắn gọn, dễ dàng thao tác và tối ưu diện tích mạch. Sử dụng công cụ "Move" để di chuyển và "Rotate" để xoay linh kiện theo nhu cầu.
- 3) Bước 3: Thiết lập các kết nối mạch Dùng công cụ "Route" để vẽ các đường mạch điện giữa các chân linh kiện, dựa theo sơ đồ nguyên lý đã thiết kế. Do vẽ mạch hai lớp, nên chúng em tận dụng lớp dưới để đi mass và các đường tín hiệu phụ, giúp giảm giao nhau và tăng độ ổn định. Ưu tiên các đường mạch thẳng, ngắn và tránh giao nhau để giảm nhiễu tín hiệu, đặc biệt là các đường UART giữa ESP32 và CP2102.
- 4) Bước 4: Tạo các via và bổ sung chi tiết phụ trợ và phủ đồng cho 2 lớp Bottom và Top Sử dụng công cụ "Via" để tạo lỗ xuyên kết nối giữa hai lớp mạch. Đồng thời bổ sung các đường cấp nguồn 3.3V, nối mass, chân header để giao tiếp với mạch ngoài, và các thông tin nhận diện như nhãn linh kiện, ký hiệu, tên mạch hoặc logo để tăng tính chuyên nghiệp và thuận tiện bảo trì. Sau đó phủ đồng (GND) cho 2 lớp bottom và top.
- 5) Bước 5: Rà soát và xác thực toàn bộ thiết kế Sau khi hoàn tất bố trí và kết nối, sử dụng tính năng kiểm tra lỗi DRC (Design Rule Check) để phát hiện các lỗi như đường mạch chồng chéo, linh kiện va chạm hoặc kết nối sai. Đồng thời xác nhận các thông số kỹ thuật như kích thước bảng mạch, độ dày lớp đồng, kích thước lỗ khoan... đều đạt yêu cầu trước khi xuất file sản xuất (Gerber) phục vụ gia công công nghiệp.

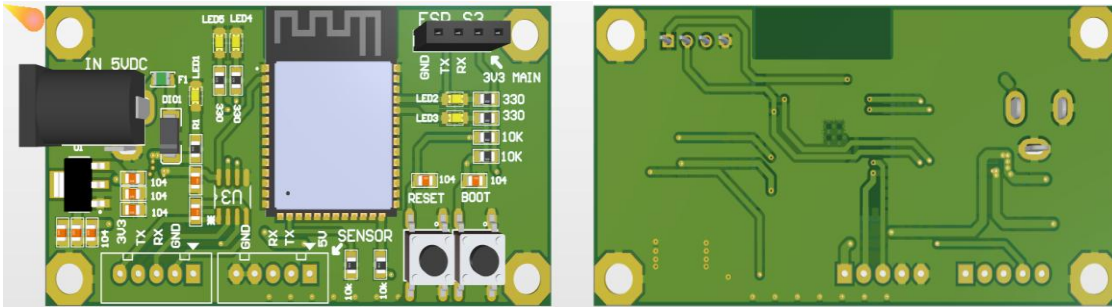
Kết quả:



Hình 3.2.1: lớp Bottom sau khi đã phủ GND (2D layout)



Hình 3.2.2: lớp Top sau khi đã phủ GND (2D layout)



Hình 3.2.3: 3D layout

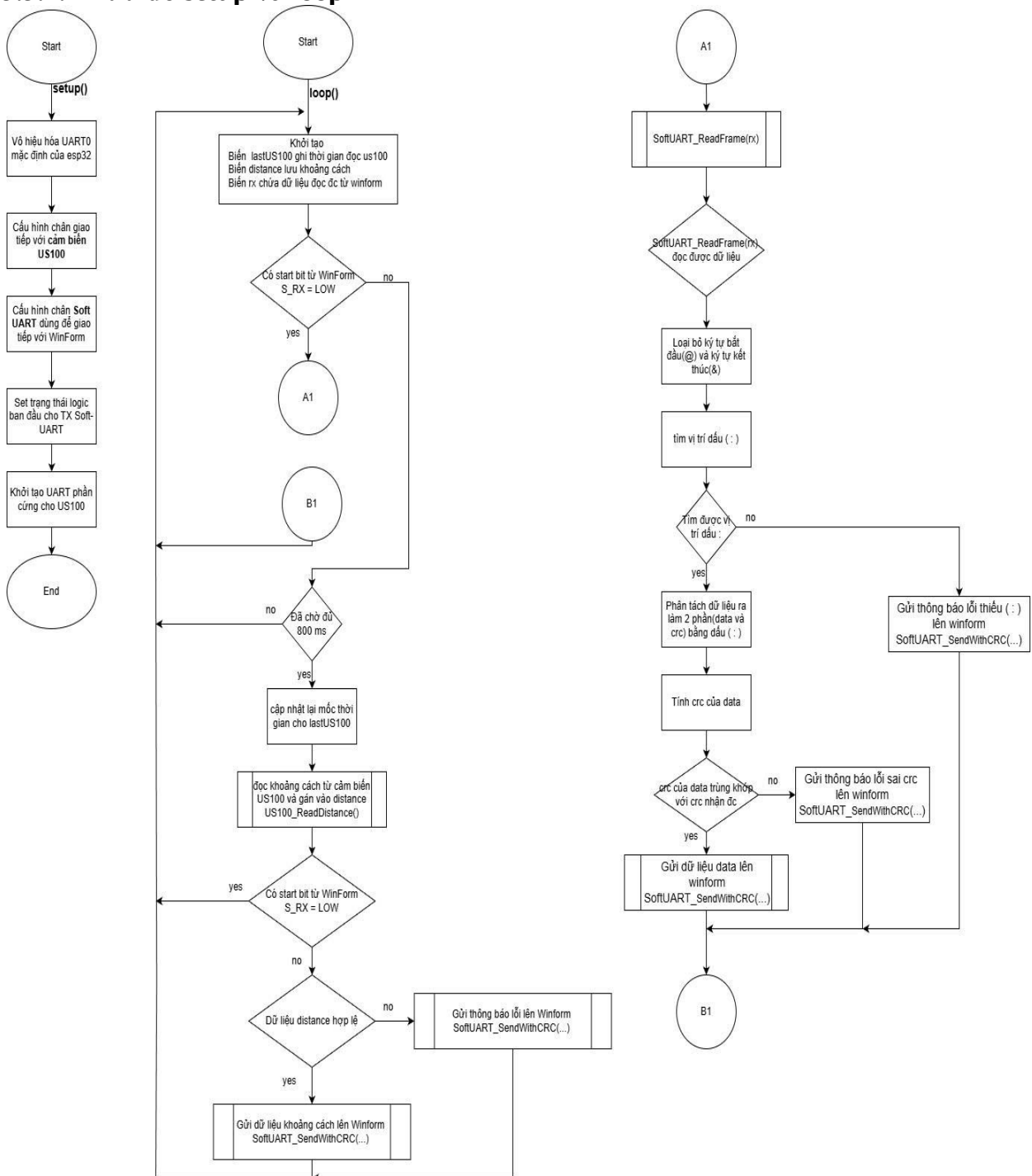
3.3 Thiết kế giải thuật và giao diện

3.3.1 Lưu đồ giải thuật Firmware

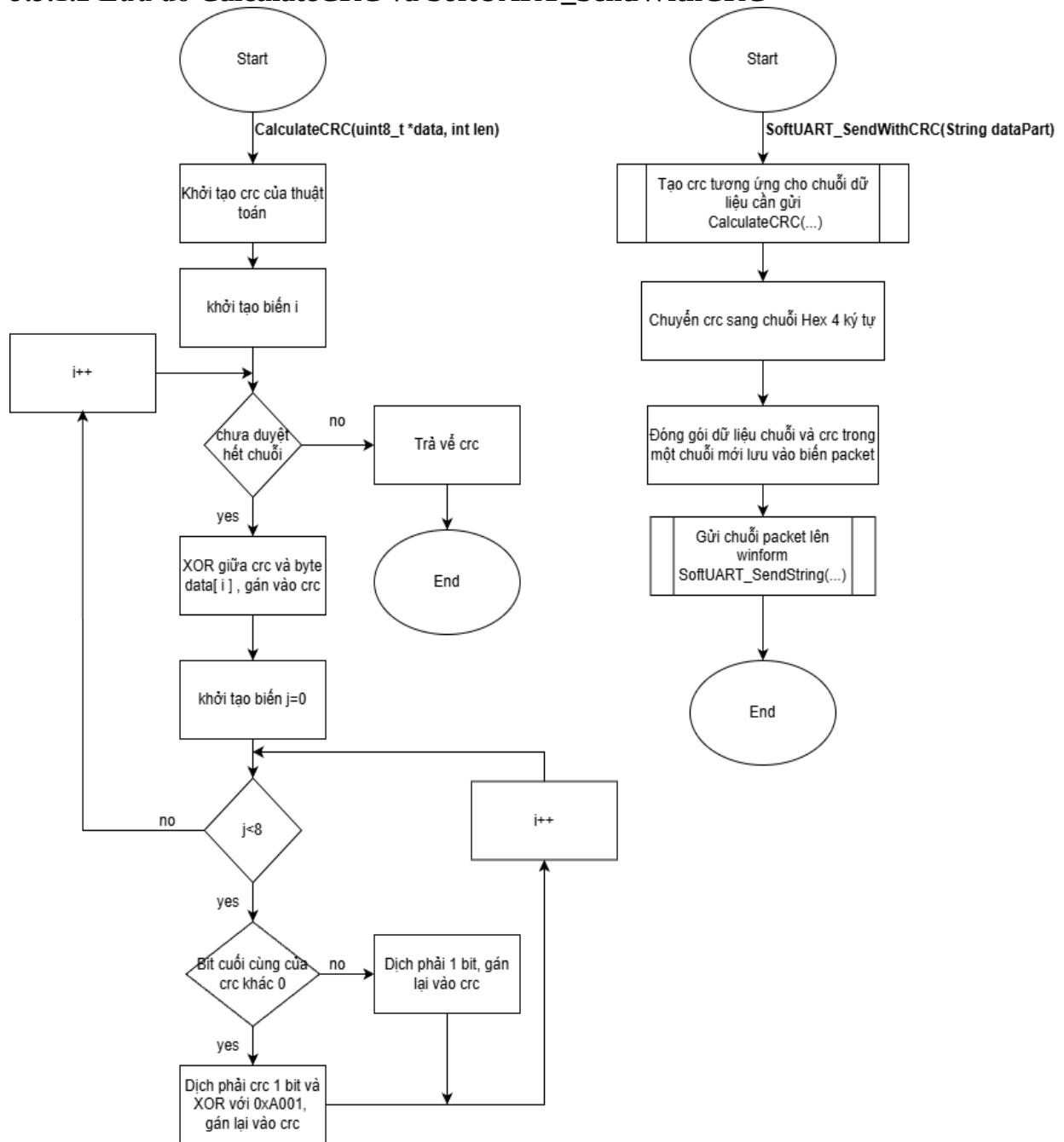
Trong quá trình xây dựng hệ thống đo khoảng cách, việc thiết kế giải thuật cho firmware đóng vai trò then chốt và cực kì quan trọng, giúp đảm bảo sự phối hợp, liên kết nhịp nhàng giữa các khối phần cứng và phần mềm. Giải thuật được xây dựng nhằm mô tả rõ ràng các bước xử lý dữ liệu từ cảm biến US-100, truyền thông qua vi điều khiển ESP32-S3, tín hiệu cách ly bằng ADuM1201 và giao tiếp với máy tính thông qua module CP2102. Việc biểu diễn giải thuật dưới dạng lưu đồ không chỉ giúp chúng em hình dung trực quan, rõ nét quá trình hoạt động của hệ thống mà còn hỗ trợ kiểm tra, tối ưu và giảm thiểu sai sót trong quá trình lập trình. Các lưu đồ dưới đây thể hiện chi tiết từng bước xử lý của firmware, từ những

việc khởi tạo, thu thập dữ liệu, xử lý tín hiệu cho đến truyền kết quả lên phần mềm WinForms C#.

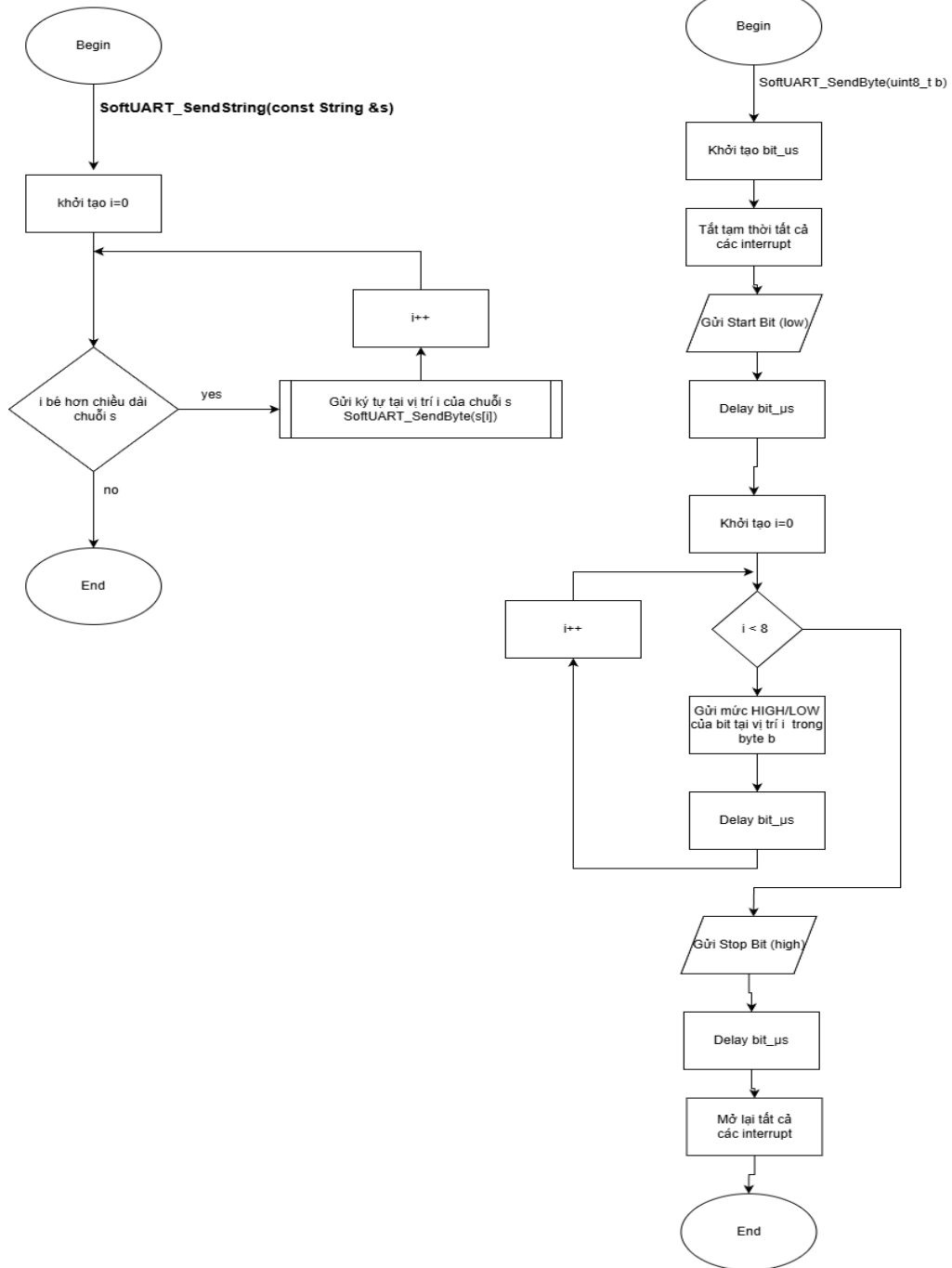
3.3.1.1 Lưu đồ setup và loop



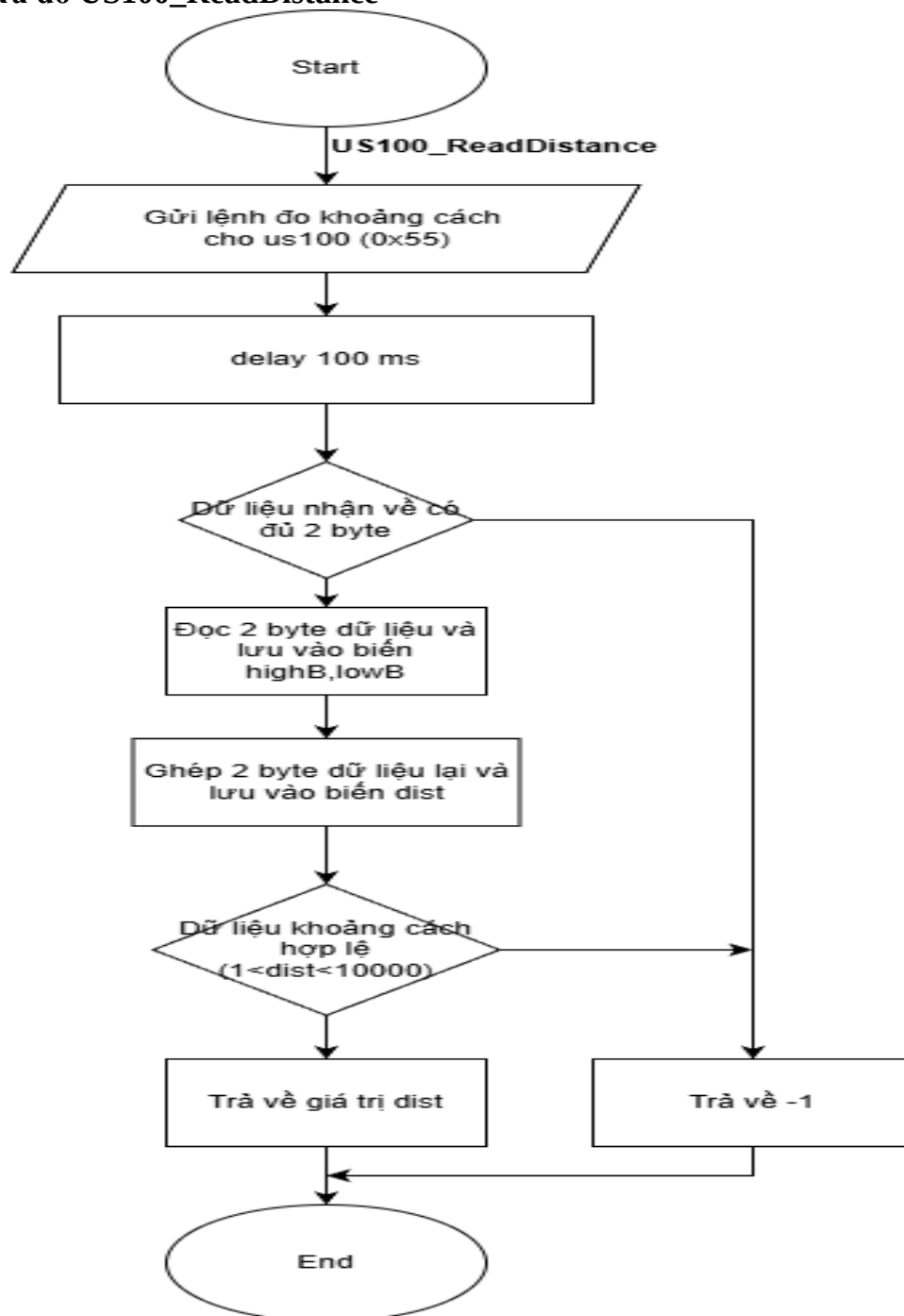
3.3.1.2 Lưu đồ CalculateCRC và SoftUART_SendWithCRC



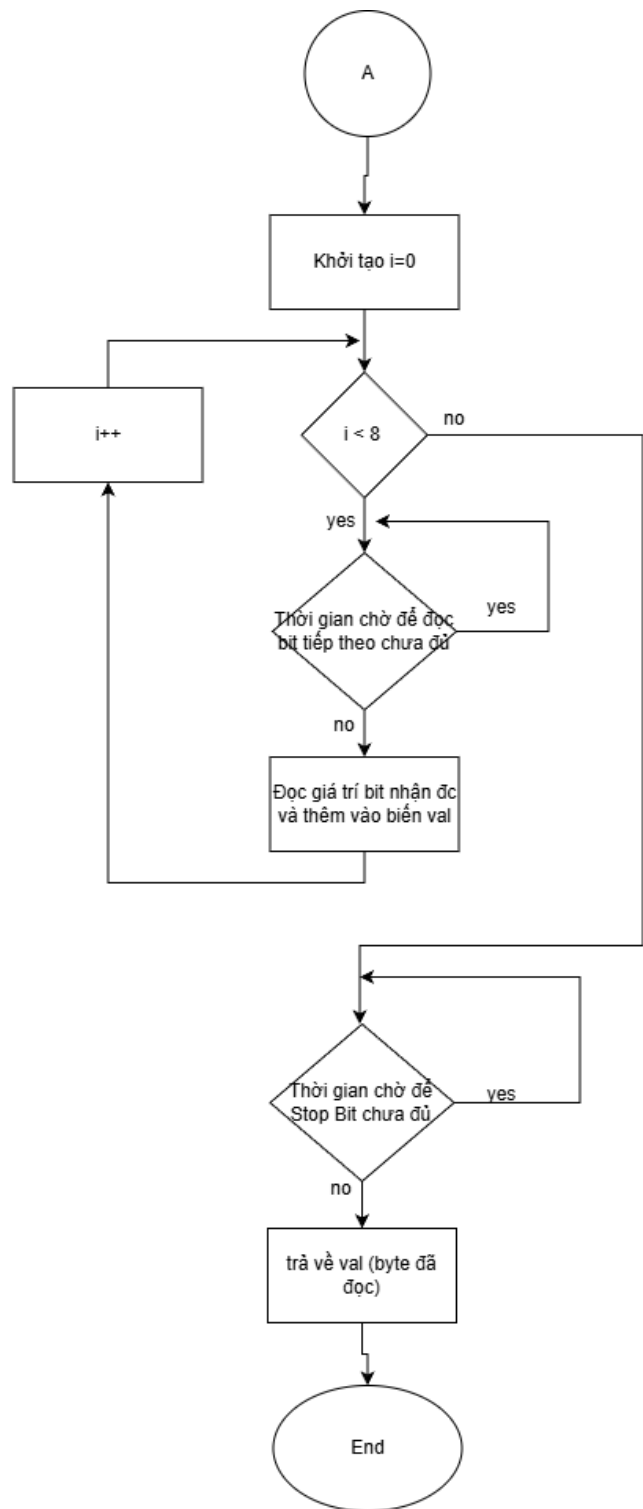
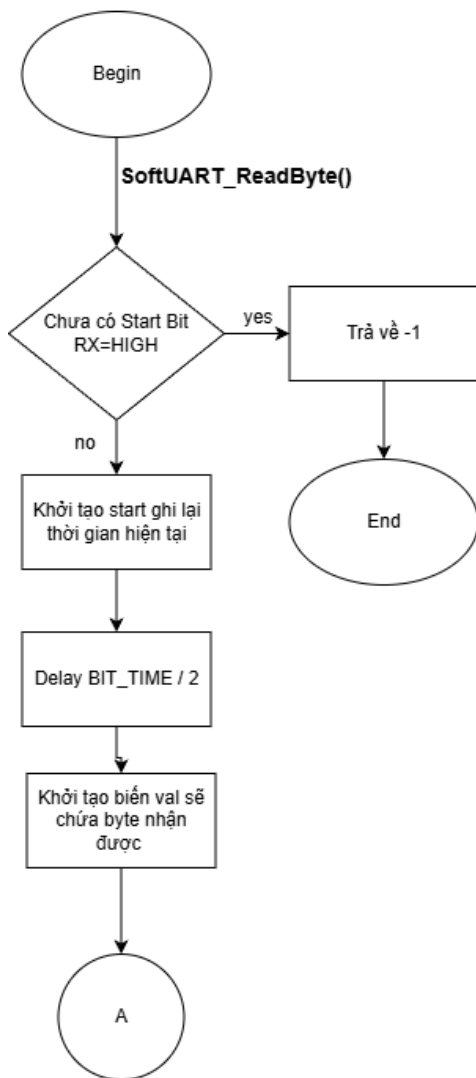
3.3.1.3 Lưu đồ SoftUART_SendByte và SoftUART_SendString



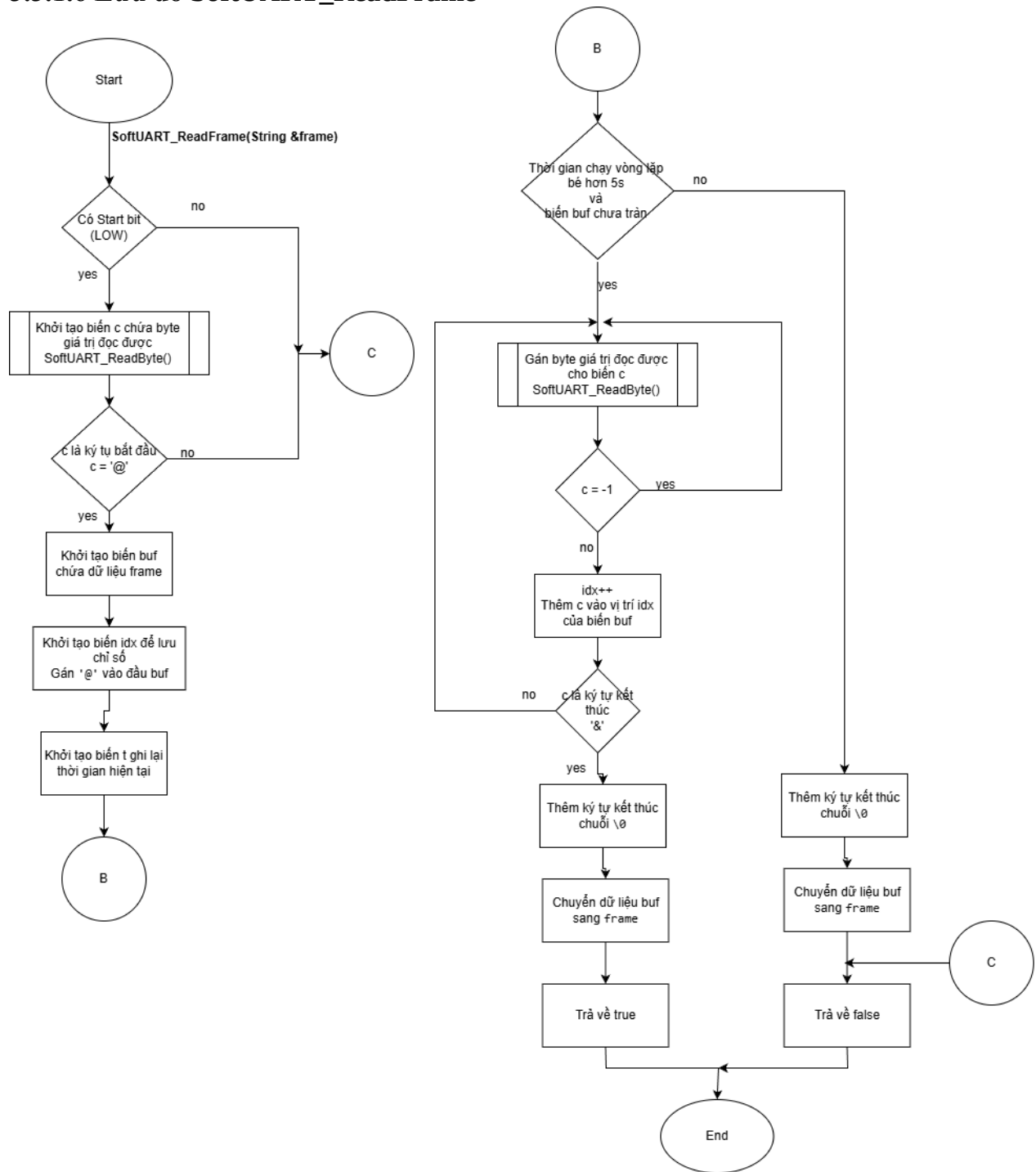
3.3.1.4 Lưu đồ US100_ReadDistance



3.3.1.5 Lưu đồ SoftUART_ReadByte



3.3.1.6 Lưu đồ SoftUART_ReadFrame



3.3.2 Thiết kế giao diện người dùng và lưu đồ giải thuật Winform C#

Để hỗ trợ người dùng trong việc giám sát và kiểm tra hoạt động của cảm biến siêu âm US-100, đề tài xây dựng một giao diện điều khiển bằng WinForms C#. Giao diện có bố cục rõ ràng, dễ sử dụng, và tích hợp đầy đủ các chức năng cần thiết cho việc truyền nhận dữ liệu qua UART mềm.

Mục tiêu thiết kế:

- Tạo môi trường tương tác trực quan giữa người dùng và hệ thống phần cứng.
- Hiển thị dữ liệu đo đạc, trạng thái truyền thông, và phản hồi từ vi điều khiển một cách rõ ràng.
- Hỗ trợ kiểm tra độ chính xác của giao tiếp UART mềm thông qua chức năng kiểm tra tự động và kiểm tra thủ công.
- Hạn chế sử dụng nhiều thư viện, câu lệnh phức tạp vì chưa có kết hợp Database

Các thành phần chính của giao diện:

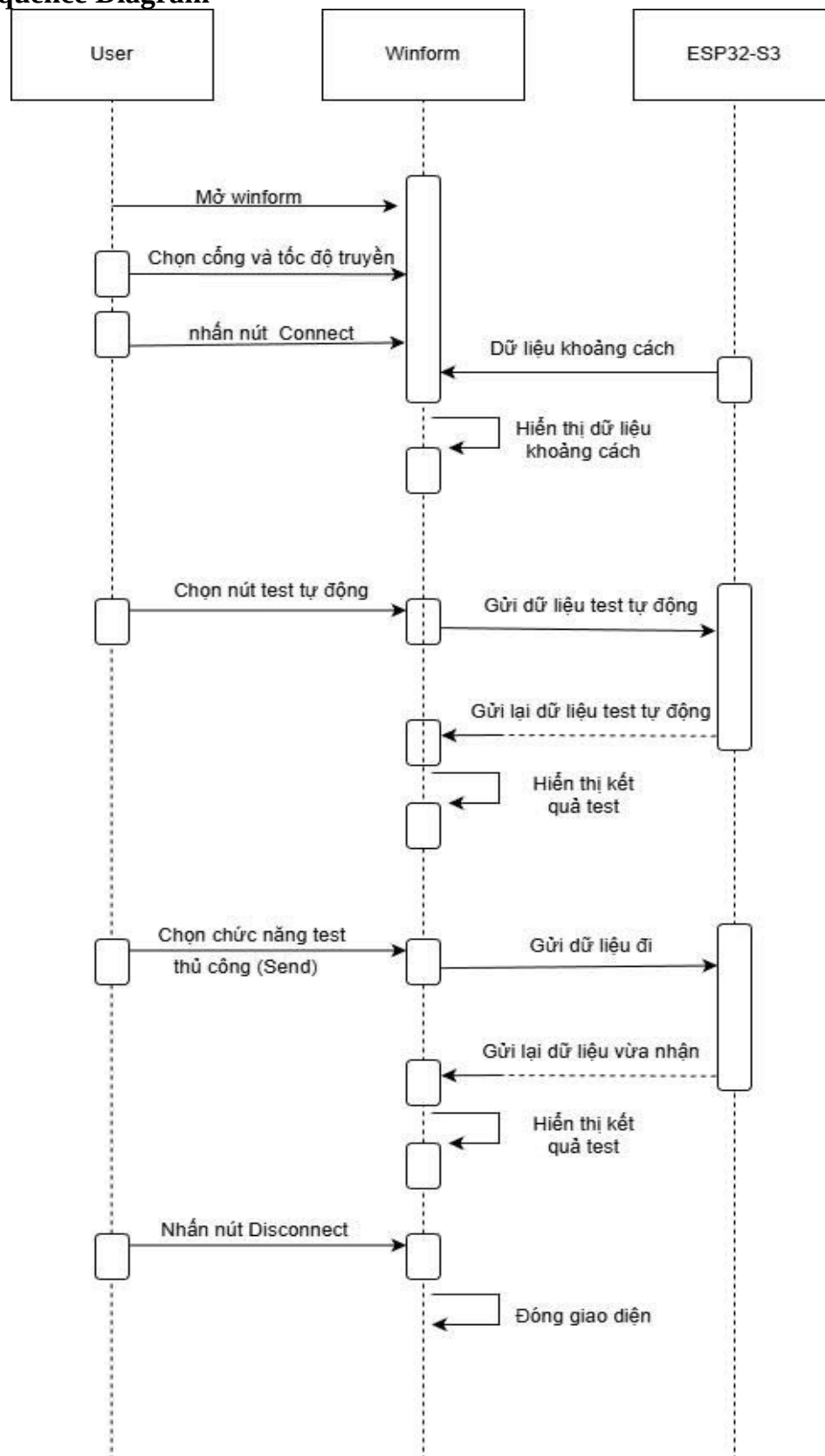
- Thiết lập truyền thông (Soft-UART):
 - Cho phép lựa chọn cổng COM và tốc độ Baud.
 - Các nút *Connect* và *Disconnect* giúp quản lý kết nối với vi điều khiển.
 - Màu sắc và trạng thái nút phản ánh tình trạng kết nối hiện tại.
- Khu vực cảm biến US-100:

- Hiển thị giá trị khoảng cách đo được từ cảm biến.
- Dữ liệu được cập nhật định kỳ hoặc theo yêu cầu kiểm tra.
- Khung kiểm tra dữ liệu:
 - Cho phép gửi dữ liệu thủ công hoặc kích hoạt chế độ *Test tự động*.
 - Chức năng *Test tự động* và *Test thủ công* giúp đánh giá độ chính xác của UART mềm bằng cách gửi và nhận chuỗi dữ liệu mẫu, sau đó so sánh kết quả.
- Thanh trạng thái và hiệu suất:
 - Các thanh tiến trình như *SEND*, *RECEIVE*, và *Accuracy* biểu diễn hiệu suất truyền nhận và độ tin cậy của dữ liệu.
 - Giúp người dùng đánh giá chất lượng truyền thông một cách trực quan.
- Nút thoát chương trình:
 - Cho phép người dùng kết thúc ứng dụng một cách an toàn.

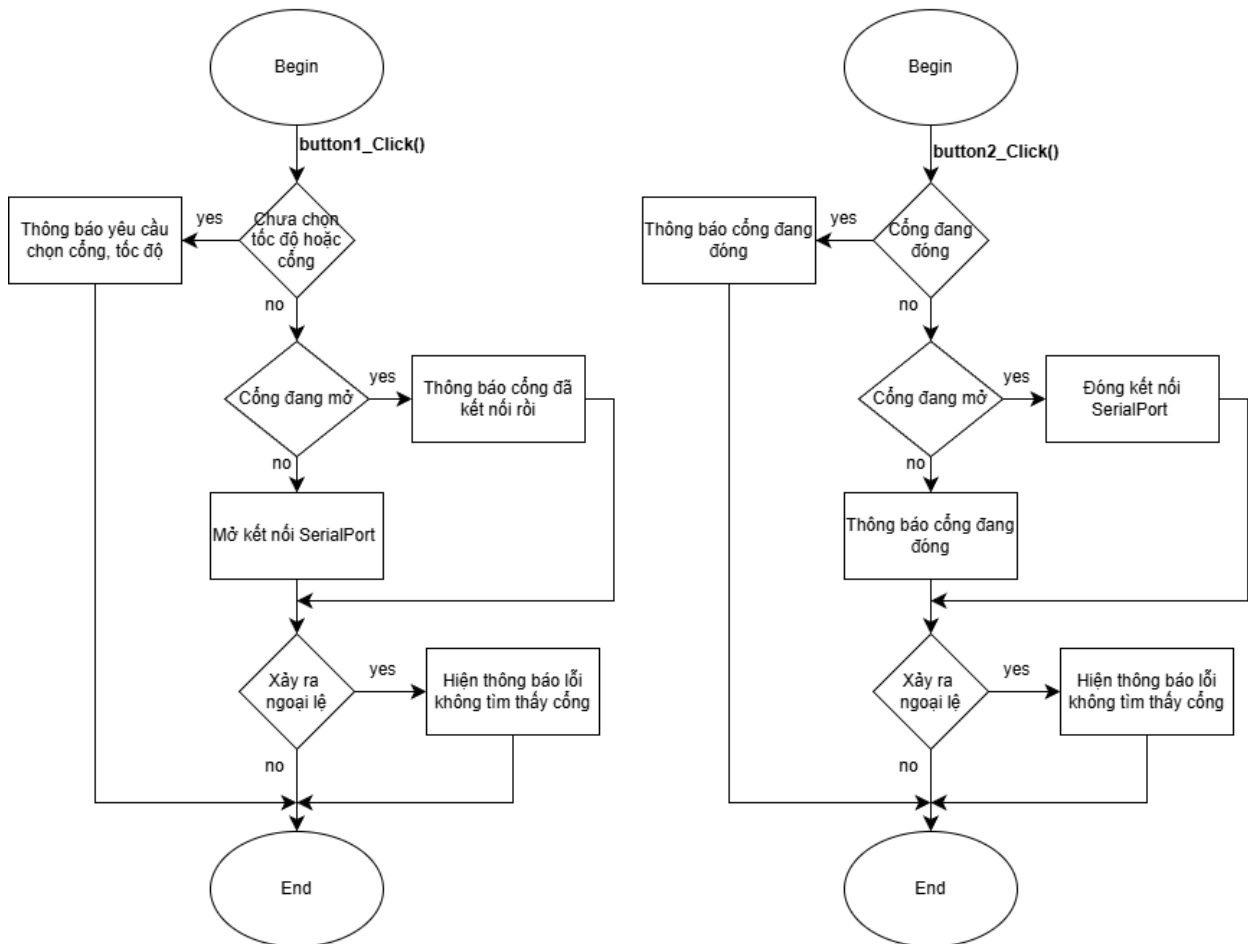
Công nghệ và kĩ thuật được dùng:

- Giao diện được xây dựng bằng WinForms C#, sử dụng các control như ComboBox, Button, TextBox, ProgressBar.
- Giao tiếp UART thực hiện qua thư viện System.IO.Ports, hỗ trợ truyền nhận dữ liệu theo định dạng ASCII.
- Dữ liệu được đóng gói theo cấu trúc rõ ràng, dễ phân tích và kiểm tra.
- Sử dụng kĩ thuật kiểm tra lỗi (CRC): Để đảm bảo dữ liệu không bị sai lệch trong quá trình truyền

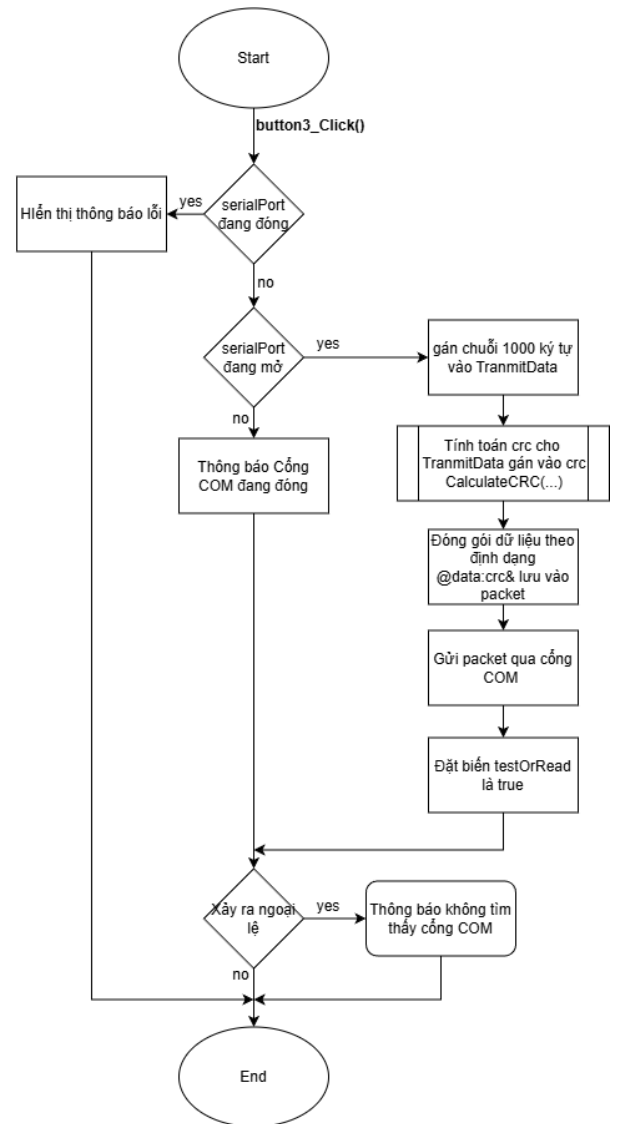
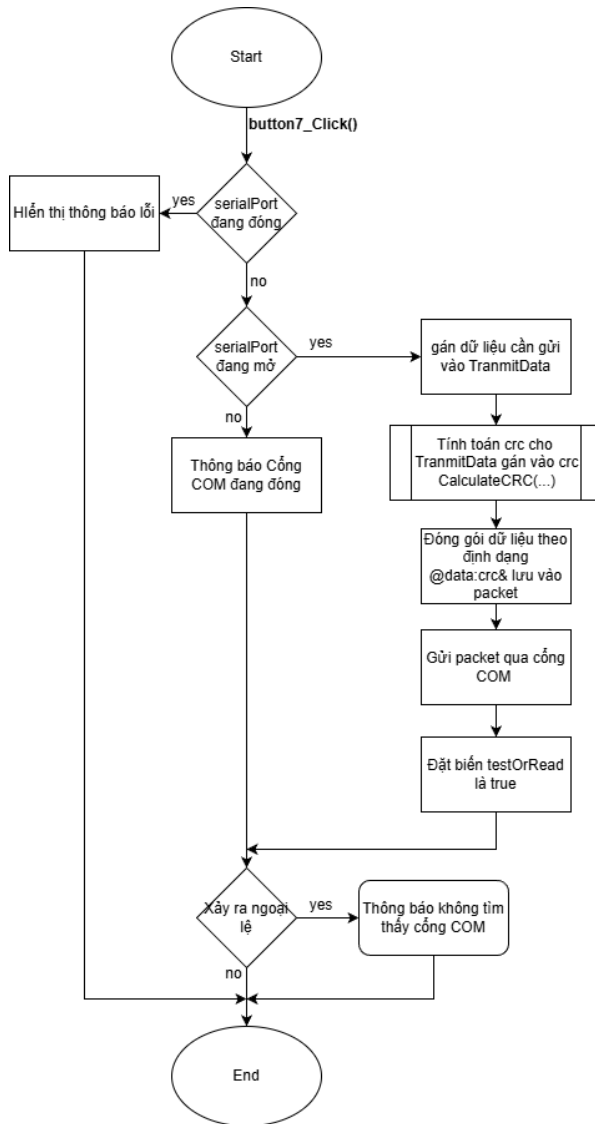
3.3.2.1 Sequence Diagram



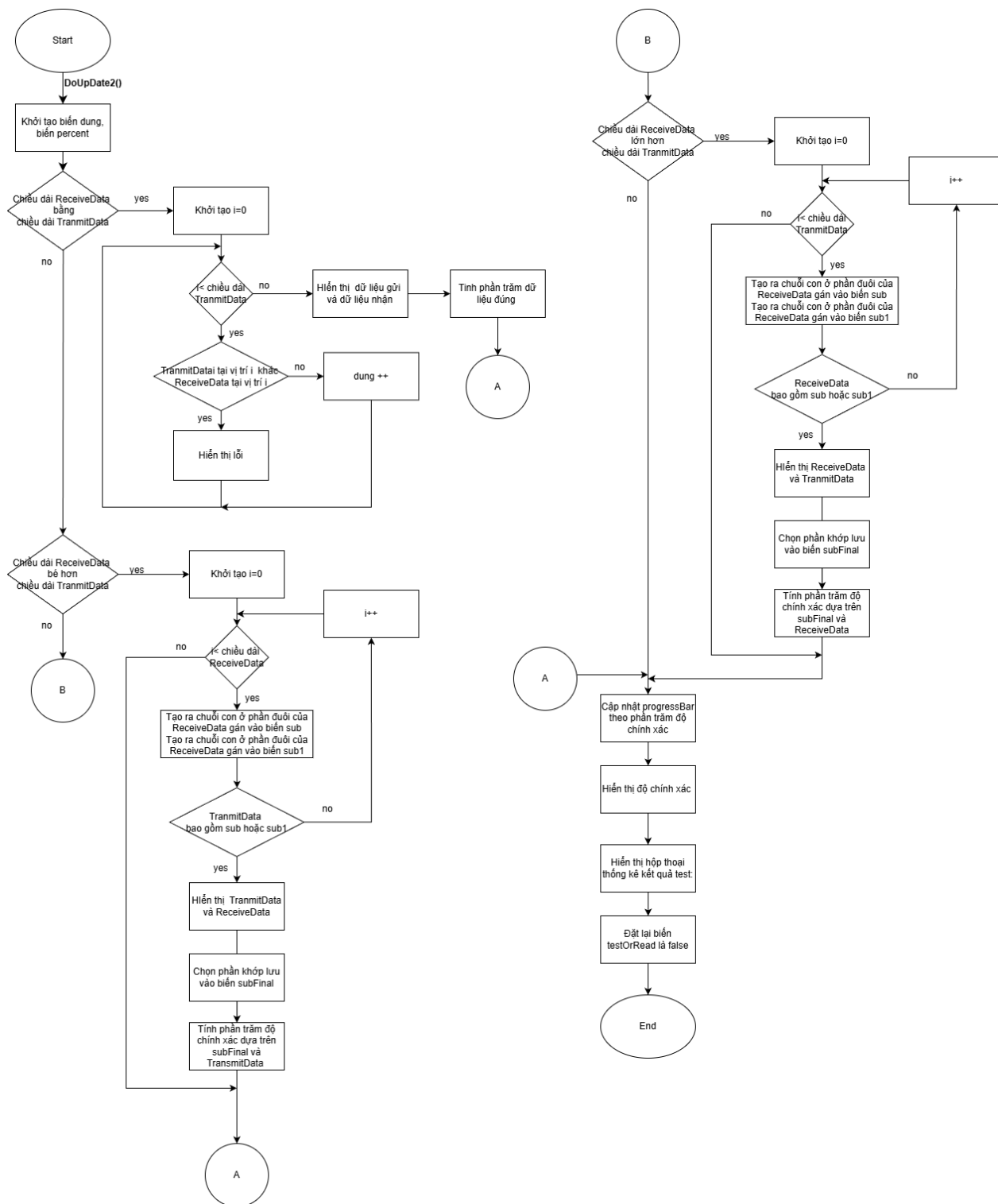
3.3.2.2 Lưu đồ Connect và Disconnect



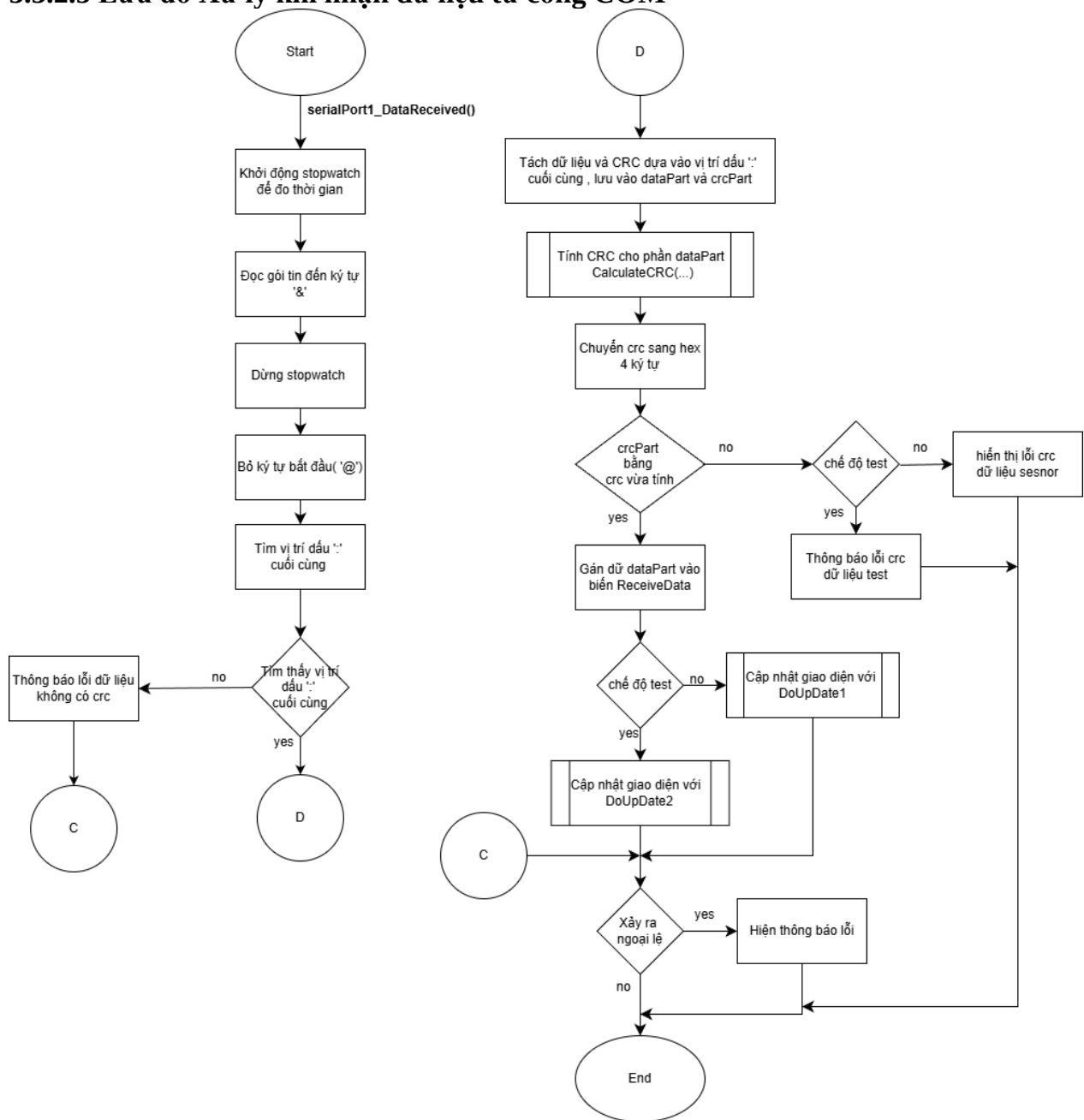
3.3.2.3 Lưu đồ Test thủ công và Test tự động



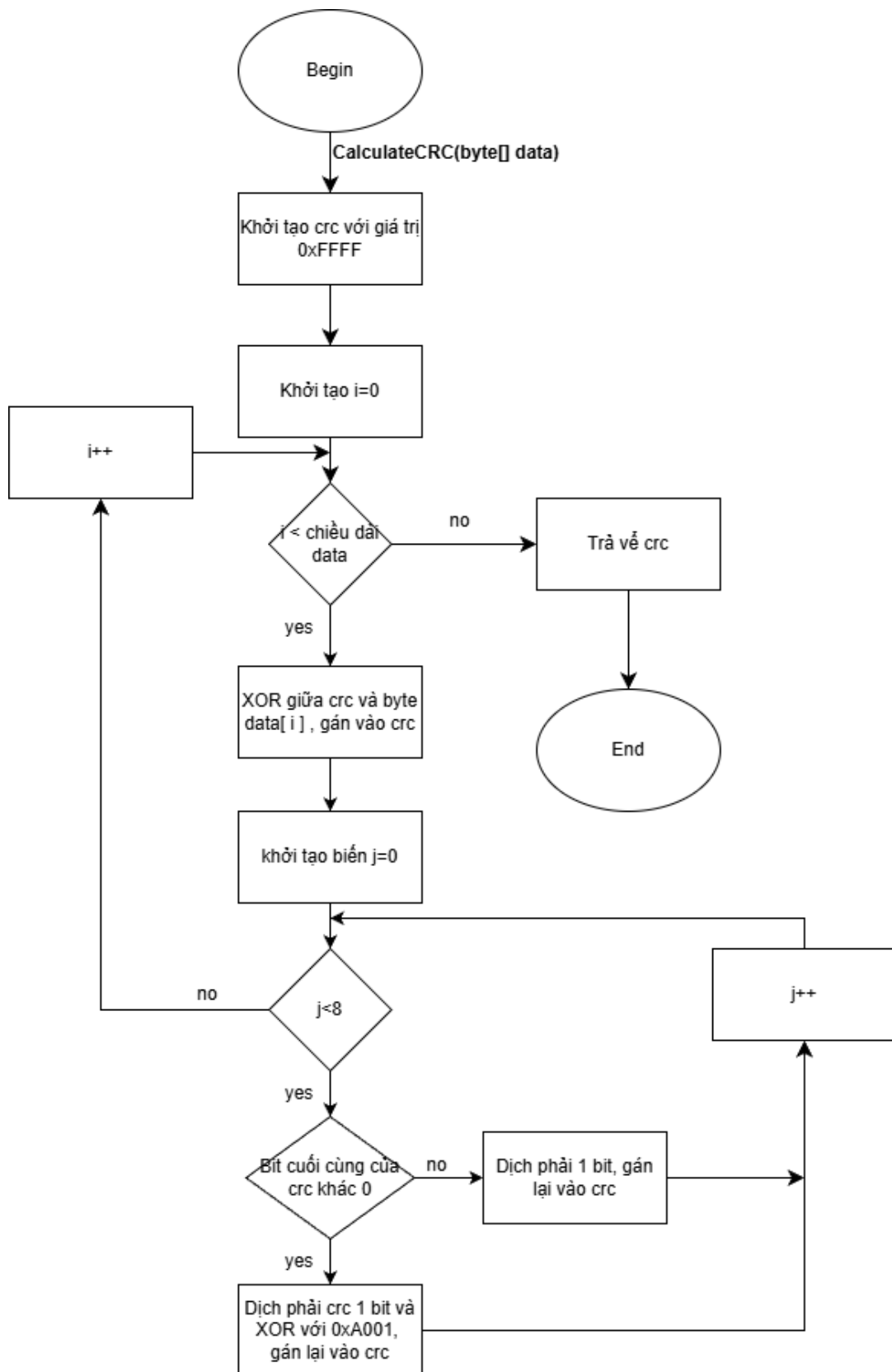
3.3.2.4 Lưu đồ Xử lý dữ liệu ở chế độ test



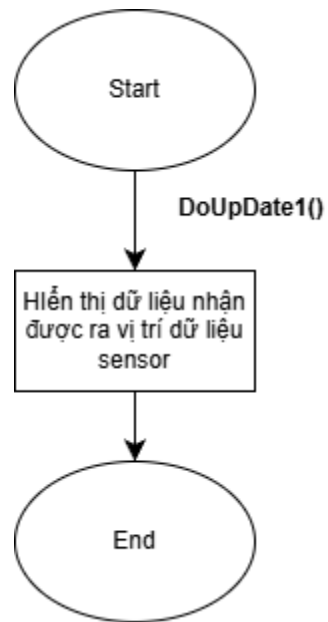
3.3.2.5 Lưu đồ Xử lý khi nhận dữ liệu từ cổng COM



3.3.2.6 Lưu đồ CalculateCRC



3.3.2.7 Lưu đồ Xử lí dữ liệu khi ở chế độ đọc cảm biến



CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Giới thiệu mục tiêu chính của thực nghiệm

4.1.1 Mục tiêu chính của thực nghiệm

Phần thực nghiệm và đánh giá kết quả là một nội dung quan trọng trong quá trình triển khai đề tài. Đây không chỉ là bước kiểm chứng tính khả thi của các thiết kế đã đề xuất, mà còn là cơ sở để đánh giá mức độ hoàn thành so với mục tiêu ban đầu. Thông qua việc thực hiện các thí nghiệm, đo đạc, và phân tích dữ liệu, nhóm thực hiện có thể xác định được tính chính xác, độ tin cậy và hiệu quả của các giải pháp kỹ thuật đã áp dụng. Trong phần này, các kết quả thực tế sẽ được trình bày một cách trực quan thông qua hình ảnh, bảng số liệu, biểu đồ và các minh chứng cụ thể. Việc đối chiếu giữa thiết kế lý thuyết và kết quả thực tế giúp làm rõ sự tương đồng hoặc khác biệt, từ đó rút ra nhận xét khách quan về chất lượng của đề tài.

Mục tiêu của phần thực nghiệm nhằm kiểm chứng tính đúng đắn của các thiết kế đã trình bày ở Chương 3, bao gồm:

- Xác nhận khả năng hoạt động ổn định của hệ thống phần cứng gồm ESP32 S3, cảm biến US100, khối cách ly ADUM1201 và nguồn 3.3V
- Đánh giá chất lượng truyền thông UART, gồm UART cứng với cảm biến và UART mềm ruyền về máy tính.
- Kiểm tra các thông số tín hiệu điện như điện áp, dạng sóng, timing, dòng tiêu thụ để đảm bảo hệ thống đáp ứng yêu cầu kỹ thuật.
- Xác thực dữ liệu thực tế từ cảm biến màu và so sánh với lý thuyết để đánh giá độ chính xác.
- Đánh giá độ tin cậy của hệ thống khi hoạt động liên tục và trong nhiều điều kiện khác nhau.
- Thực hiện kiểm tra truyền nhận dữ liệu 1000 ký tự để đánh giá mức độ tin cậy của UART mềm.
- So sánh kết quả thực nghiệm với tính toán lý thuyết và các sơ đồ thiết kế nhằm xác nhận tính hợp lý của phương án triển khai.

Sau quá trình Hàn linh kiện, mạch phần cứng trên thực tế đã được hoàn thiện. Để hình thành dự án này hệ thống đã được lắp ráp bao gồm board mạch chính điều khiển, module cảm biến

US100 và khối giao tiếp với máy tính đồng thời cũng là khối nạp chương trình. Hình 4.1 dưới đây mô tả chi tiết cách bố trí và kết nối các thành phần trong thực tế:



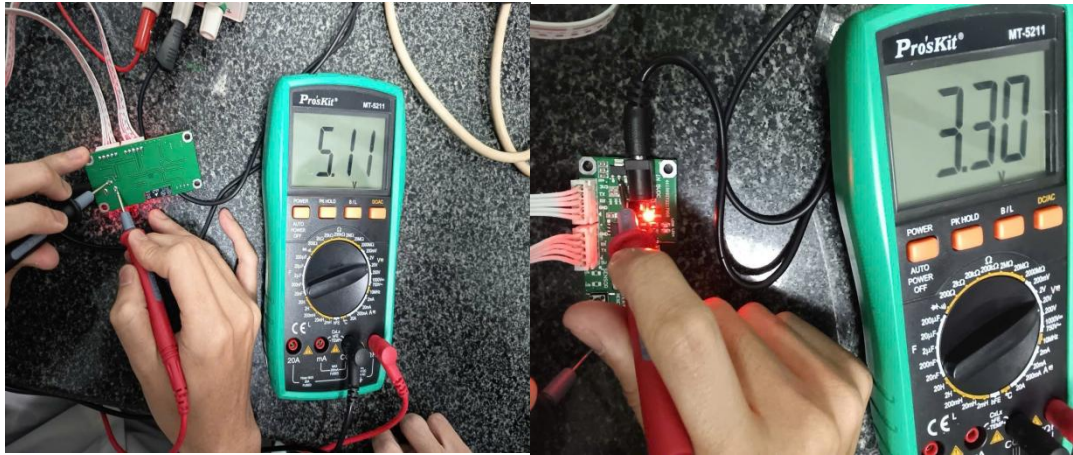
Hình 4.1: Mô hình Board mạch thực tế

4.2 Kiểm tra và đo đặc phần cứng

Để đảm độ ổn định và tin cậy của hệ thống trước khi đưa vào vận hành trong thực tế, việc kiểm thử, đo đặc tổng thể các trị số cơ bản như nguồn, dòng điện tiêu thụ, dáng sóng của tín hiệu...là bước bắt buộc phải có. Nhóm thực hiện đã sử dụng máy hiện sóng (máy hiện sóng- Oscilloscope) để quan sát trực tiếp dạng sóng tại các chân giao tiếp giữa cảm biến và vi điều khiển cũng như chân giao tiếp Uart mềm với máy tính. Bản chất của quá trình này kiểm tra độ rộng xung của các chân tín hiệu (Uart cứng và Uart mềm) và đánh giá mức độ nhiễu tác động lên đường truyền tín hiệu.

4.2.1 Đo nguồn, dòng tiêu thụ và đánh giá

Bước kiểm tra điện áp nguồn cấp là bắt buộc để đảm bảo an toàn cho các linh kiện. Hình 4.2.1.1 hiển thị bo mạch được cấp nguồn đầu vào khoảng 5.11V. Sau khi được IC ổn áp tuyến tính SMD 1117-3.3 ổn áp, điện áp ngõ ra sau khi qua các tụ lọc nhiễu đo được trên đồng hồ vạn năng là 3.3V. Giá trị này nằm trong ngưỡng được sai cho phép ($\pm 5\%$) so với mức lý thuyết 3.3V. Điều này đã chứng minh rằng khối nguồn hoạt động ổn định và đủ điều kiện cấp cho vi điều khiển.

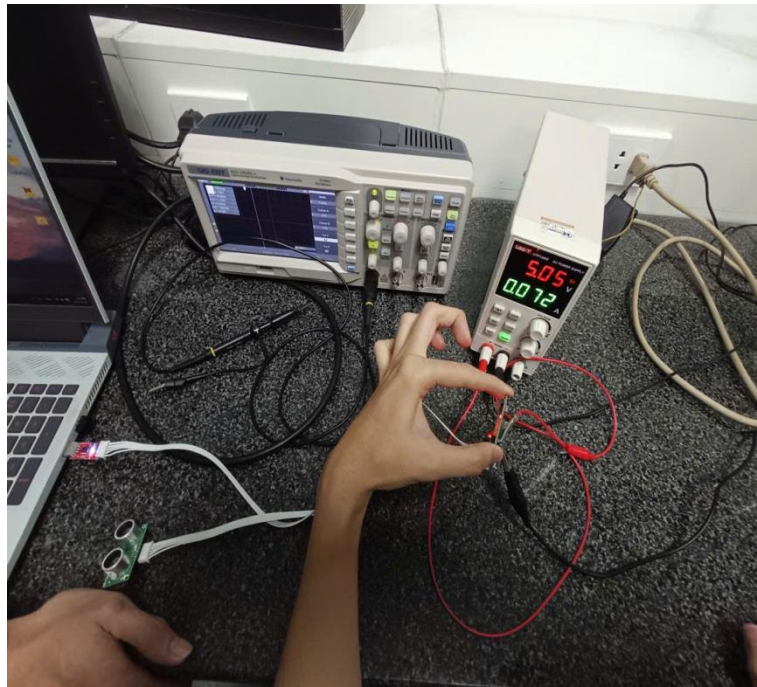


Hình 4.2.1.1: Kiểm thử mạch nguồn

Bước tiếp theo chúng em kiểm tra được dòng tiêu thụ ở trạng thái chờ và dòng tiêu thụ ở trạng thái hoạt động tất cả các chức năng. Để đo được dòng tiêu thụ ở hai trạng thái ta thực hiện nhờ tính năng đo trực tiếp của máy cấp nguồn. Hình 4.2.1.2 cho thấy rõ được dòng tiêu thụ ở trạng thái chờ đo được là 0.054A (54mA). Hình 4.2.1.3 cho thấy rõ dòng tiêu thụ ở trạng thái hoạt động tất cả các chức năng 0.072A (72mA). Sau khi đo chúng em có thể tự tin khẳng định rằng đây là mức dòng tiêu thụ tiêu vi điều khiển và cho thấy các module cảm biến khi đang hoạt động bình thường. Giá trị này đã góp phần chứng minh mạch không xảy ra hiện tượng ngắn mạch, hở mạch là yếu tố quyết định để đảm bảo các linh kiện đang nhận đủ dòng để vận hành.



Hình 4.2.1.2: Đo dòng ở trạng thái chờ

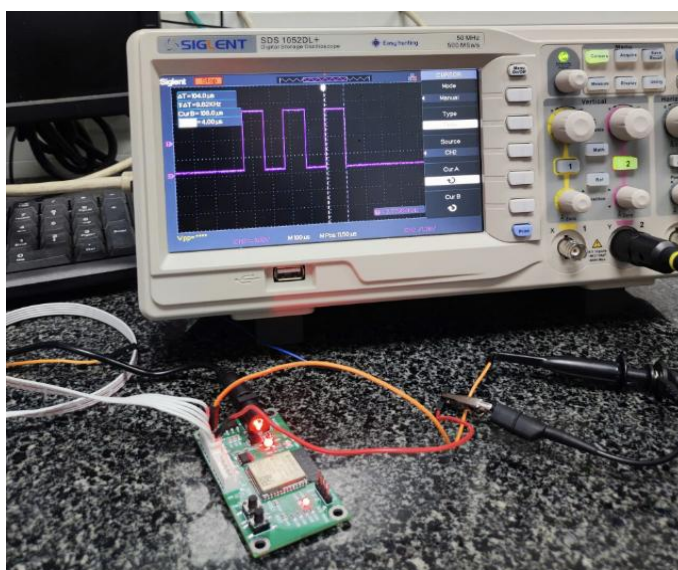


Hình 4.2.1.3 Dòng đo ở trạng thái hoạt động tắt cả chức năng

Tại lần kiểm thử, đo đạt này nhóm chúng em nhận định rằng mạch chúng em làm sẽ hoạt động tương đối tốt đã đáp ứng được các yêu cầu kỹ thuật cũng như yêu cầu cần và đủ cho đề tài này. Mạch hoạt động ổn định không có hiện tượng chậm chạp.

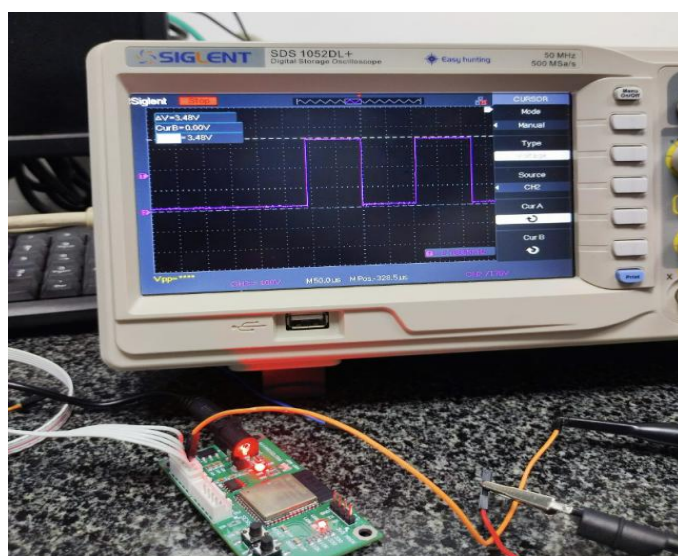
4.2.2 Đo Waveform UART mềm và đánh giá

Trong quá trình thiết kế UART mềm, tôi lựa chọn tốc độ truyền là 9600 baud. Theo lý thuyết, thời gian truyền của một bit được tính bằng công thức $T = 1/9600$, tương đương khoảng 0.000104 giây hay 104 micro giây (μs). Sau khi nạp chương trình gửi chữ A liên tục không có delay và tiến hành đo tín hiệu bằng máy hiện sóng, kết quả hiển thị cho thấy chu kỳ bit đúng bằng $\Delta T = 104\mu s$, thể hiện rõ tại hình 4.2.2.1. Điều này cho thấy tín hiệu UART mềm đã được tạo ra chính xác theo tốc độ truyền mong muốn, hoàn toàn phù hợp với tính toán lý thuyết ban đầu. Đây là minh chứng rõ ràng cho thấy việc lập trình UART mềm đã đạt yêu cầu về mặt thời gian truyền dữ liệu. Hình 4.2.2.1 cũng thể hiện rõ mức logic 0 và logic 1, tín hiệu Uart mềm ứng với gian $\Delta T = 104\mu s$ là hoàn toàn khả thi đúng với tính toán ban đầu.



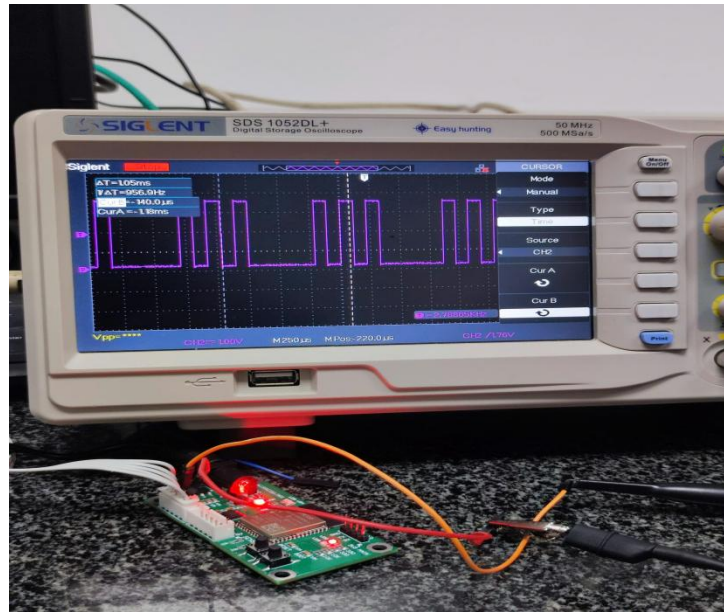
Hình 4.2.2.1: Đo điện áp bằng máy hiện sóng

Nhóm cũng tiến hành đo điện áp bằng máy hiện sóng, kết quả đo được thể hiện rõ ở hình 4.2.2.2 kết quả đo ra $\Delta V = 3.36V$ là hoàn toàn được chấp nhận với yêu cầu của thành viên đã thiết kế PCB.



Hình 4.2.2.2: Đo dây điện áp hoạt động của Uart mềm

Chứng minh dạng sóng sau khi gởi chữ “A” liên tục và đánh giá:

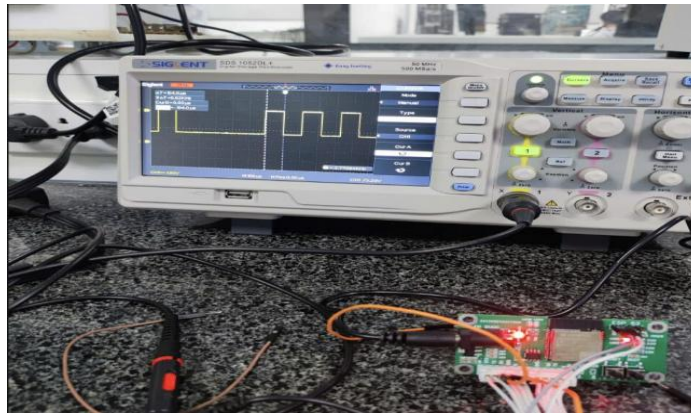


Hình 4.2.2.3: Dạng sóng khi truyền chữ A liên tục

ESP32-S3 sử dụng UART mềm để liên tục gửi ký tự ‘A’ qua chân TX. Mà chữ “A” trong bản mã ASCII là 0x41(0100 0001), khi truyền theo chuẩn UART thì theo quy ước bit thấp đi trước, nên khung truyền dữ liệu sẽ gồm có: một bit start bằng 0, tiếp theo là 8 bit dữ liệu đảo thứ tự (1000 0010), và cuối cùng là bit stop bằng 1. Như vậy toàn bộ khung truyền có 10 bit: 0 1000 0010 1, và nó được lặp lại liên tục. Kết quả đo thực tế thể hiện tại hình 4.2.2.3 cho thấy đã đúng với lý thuyết đề ra phù hợp với tốc độ 9600 baud, mỗi bit mất khoảng 104 μ s, nên cả khung mất khoảng 1.04 ms(đã minh chứng tại hình 4.2.2.1), như vậy kết quả đo thực tế cũng cho ra đúng khoảng 1.05 ms, gần như trùng khớp với lý thuyết, chứng tỏ UART mềm trên ESP32-S3 hoạt động chính xác cả về tốc độ lẫn dữ liệu.

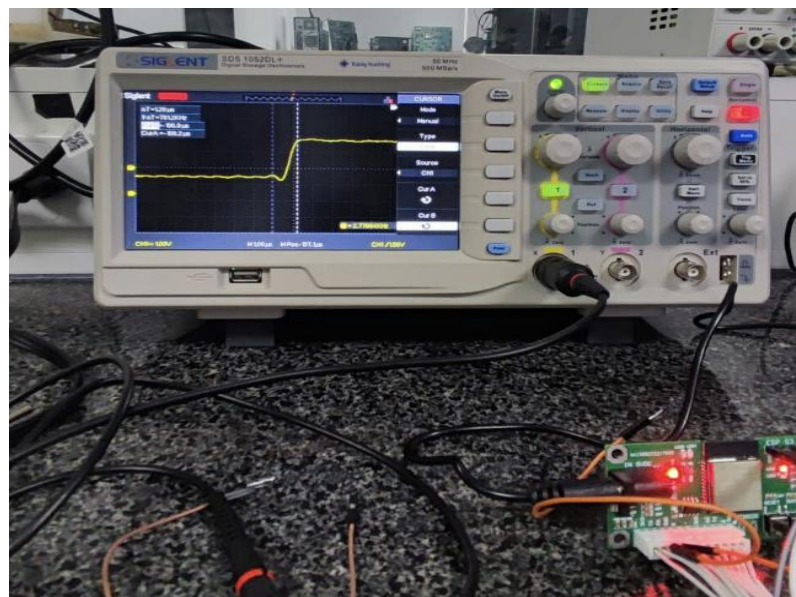
Đo thời gian T-on và T-off bằng máy hiện sóng:

Do nhóm chúng em thực hiện đo liên tục nhiều ngày và chọn ra những tấm hình có dạng sóng đẹp, trực quan dễ nhìn, dễ tính toán nên có một vài ảnh sử dụng màu vàng và tím lẫn lộn do mỗi lần đo không đồng bộ cùng một kênh. Nhóm xin khẳng định 100% là nhóm chúng em tự đo, tự kiểm tra và dạng sóng trực tiếp lấy từ board của nhóm. Hình bên dưới thể hiện đo đạt lần 2 :



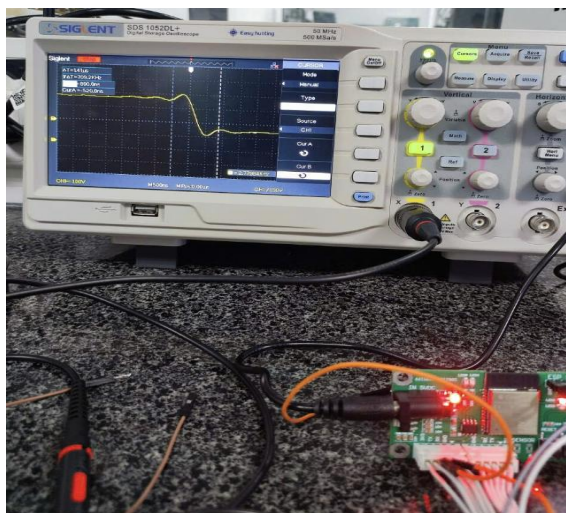
Hình 4.2.2.4: Đo thời gian T-on và T-off

Sau khi hoàn tất việc đo các thông số cơ bản như nguồn, dòng chúng em tiếp tục tiến hành đo thời gian cạnh lên (T-on) của tín hiệu UART mềm với tốc độ truyền 9600 baud. Tín hiệu UART ở trạng thái nghỉ là mức cao, và khi chuyển từ mức thấp lên mức cao để kết thúc bit START hoặc bắt đầu bit dữ liệu, sẽ tạo ra cạnh xung lên. Dựa trên dạng sóng Hình 4.2.2.4, chúng em xác định được thời gian cạnh lên là khoảng $1.29 \mu s$. để đánh giá mức độ nhanh của cạnh này, chúng em tính thời gian truyền của một bit bằng công thức: $T_{bit} = 1/9600 \approx 104.17$ Từ đó, thời gian cạnh lên chiếm khoảng 1.25% ($T = (1.29/104) * 100$) so với toàn bộ thời gian của một bit. Đây là một giá trị rất nhỏ, cho thấy cạnh lên diễn ra nhanh và sắc nét. Việc xung nhiễu T-on nhỏ hơn 2% thời gian bit cho thấy tín hiệu UART mềm được tạo ra có chất lượng tốt, đảm bảo độ chính xác khi truyền dữ liệu và hạn chế tối đa sai lệch trong quá trình lấy mẫu.



Hình 4.2.2.5: Thời gian T-on của Uart mềm

Sau đó chúng em tiến hành đo thời gian cạnh xuống T-off. Tại hình 4.2.2.5 dạng sóng thu được từ máy hiện sóng, chúng em xác định được thời gian cạnh xuống là khoảng 1.41 μ s. Để đánh giá mức độ nhanh của cạnh này, chúng em tính thời gian truyền của một bit bằng công thức: $T_{bit} = 1/9600 \approx 104.17$, Từ đó, thời gian cạnh xuống chiếm khoảng 1.35% ($T = (1.41/104) * 100$) so với toàn bộ thời gian của một bit. Đây là một giá trị rất nhỏ, cho thấy cạnh xuống diễn ra nhanh và sắc nét, đảm bảo tín hiệu rõ ràng và ổn định. Điều này chứng tỏ mạch hoạt động tốt, không bị ảnh hưởng bởi nhiễu hay trễ tín hiệu, và phù hợp với yêu cầu kỹ thuật của giao tiếp UART. Việc cả T-on và T-off (xung nhiều) đều nhỏ hơn 2% thời gian bit cho thấy tín hiệu UART mềm được tạo ra có chất lượng tốt, đảm bảo độ chính xác khi truyền dữ liệu và hạn chế tối đa sai lệch trong quá trình lấy mẫu. Điều này cũng đã thể rõ được chức năng làm việc của khối cách ly AduM1201ARZ thật sự rất tốt đạt yêu cầu mà nhóm mong muốn.

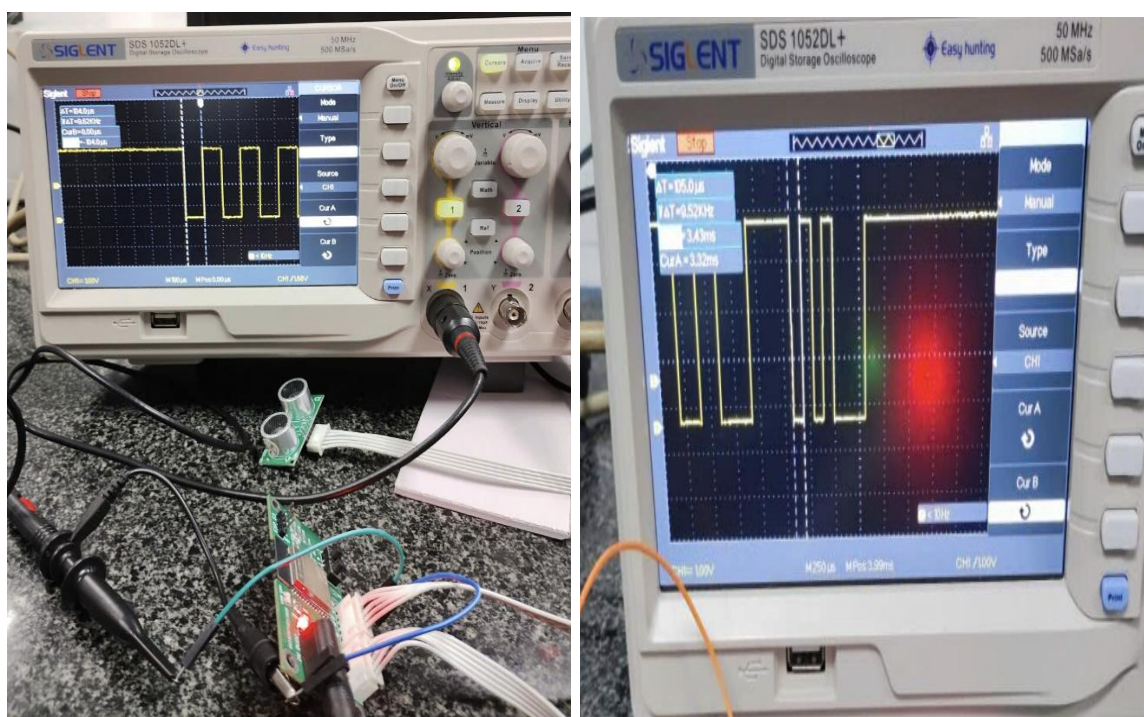


Hình 4.2.2.6: Thời gian T-off của Uart mềm

4.2.3 Đo Waveform UART US100 và đánh giá

Trong quá trình đọc dữ liệu cảm biến bằng giao thức UART, chúng em lựa chọn tốc độ truyền là 9600 baud. Theo lý thuyết, thời gian truyền của một bit được chúng em tính bằng

công thức $T=1/9600$, tương đương khoảng 0.000104 giây hay 104 micro giây (μs). Sau khi nạp chương trình và tiến hành đo tín hiệu bằng máy hiện sóng, kết quả hiển thị cho thấy chu kỳ bit đúng bằng $\Delta T = 104\mu s$, thể hiện rõ tại hình 4.2.3.1. Điều này cho thấy tín hiệu UART chính xác gần đúng theo tốc độ truyền mong muốn, hoàn toàn phù hợp với tính toán lý thuyết ban đầu. Đây là minh chứng rõ ràng cho tín hiệu UART của US100 đã đạt yêu cầu về mặt thời gian truyền dữ liệu. Hình 4.2.3.1 cũng thể hiện rõ *mức logic 0 và logic 1*, và cũng khẳng định rằng thời gian T-on và T-off của tín hiệu Uart mềm ứng với gian $\Delta T = 104\mu s$ là hoàn toàn khả thi đúng với tính toán ban đầu.



Hình 4.2.3.1: Dạng sóng UART Của US100 phụ thuộc vào
Khoảng cách đo được

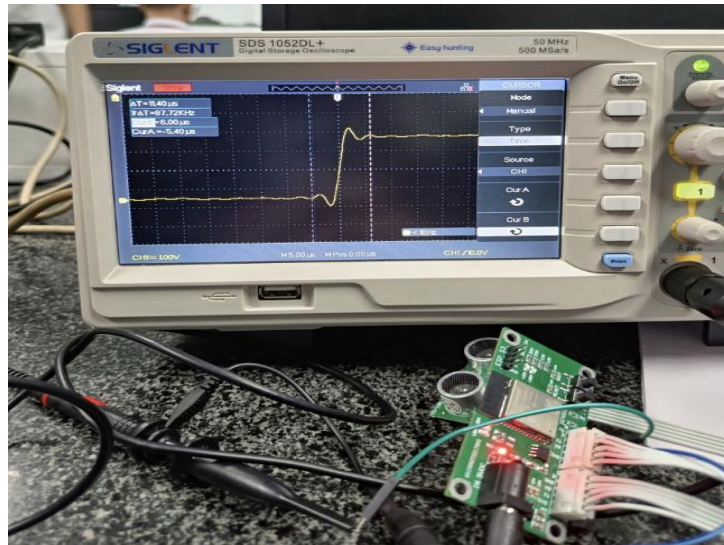


Hình 4.2.3.2: Dùng máy hiện sóng đo điện áp

Nhóm cũng tiến hành đo điện áp bằng máy hiện sóng, sau khi rung to xung ra để dễ dàng quan sát, kết quả đo được thể hiện rõ ở hình 4.2.3.2 kết quả đo ra $\Delta V = 5V$ là hoàn toàn được chấp nhận với yêu cầu của thành viên đã thiết kế PCB.

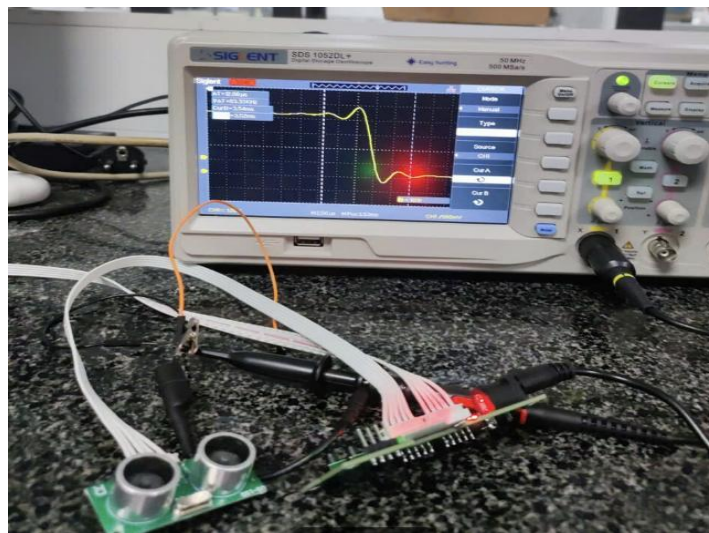
Đo thời gian T-on và T-off bằng máy hiện sóng:

Chúng em tiếp tục tiến hành đo thời gian cạnh lên (T-on) của tín hiệu UART với tốc độ truyền 9600 baud. Tín hiệu UART ở trạng thái nghỉ là mức cao, và khi chuyển từ mức thấp lên mức cao để kết thúc bit START hoặc bắt đầu bit dữ liệu, sẽ tạo ra cạnh xung lên. Dựa trên dạng sóng Hình 4.2.3.3, chúng em xác định được thời gian cạnh lên là khoảng $\Delta T = 11.40\mu s$. Để đánh giá mức độ nhanh của cạnh này, chúng em tính thời gian truyền của một bit bằng công thức tính toán như các bước trên: $T_{bit} = 1 / 9600 \approx 104\mu s$. Từ đó, thời gian cạnh lên chiếm khoảng: $T = (11.40 / 104) \times 100 \approx 10.94\%$ so với toàn bộ thời gian của một bit.



Hình 4.2.3.3: T-on US100

Tỷ lệ này cho thấy rằng quá trình chuyển mức từ thấp lên cao diễn ra chậm hơn so với các cạnh lý tưởng thường thấy trong truyền thông UART. Dù vẫn đảm bảo hoạt động ổn định ở tốc độ 9600 baud, việc cạnh lên chiếm gần 11% thời gian của một bit có thể làm giảm khoảng cách an toàn khi lấy mẫu, đặc biệt trong điều kiện tín hiệu bị nhiễu hoặc xuất hiện dao động không mong muốn. Nhưng vẫn có thể chấp nhận được.



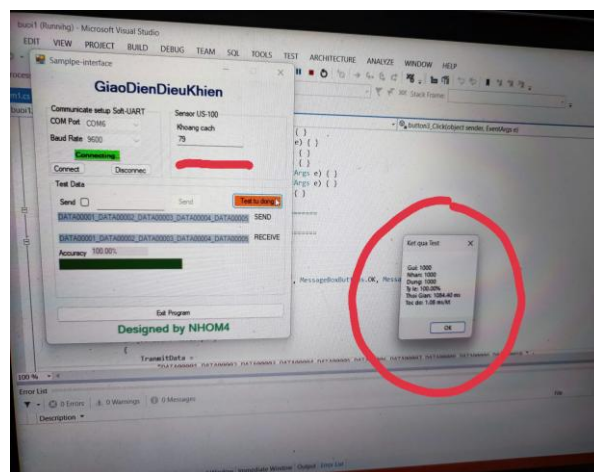
Hình 4.2.3.4: T-off US100

Dựa trên dạng sóng Hình 4.2.3.4, chúng em xác định được thời gian cạnh xuống (T-off) của tín hiệu UART là khoảng $\Delta T = 10.9\mu s$. Với tốc độ truyền 9600 baud, thời gian truyền

của một bit được tính bằng công thức: $T_{bit} = 1 / 9600 \approx 104\mu s$. Từ đó, thời gian cạnh xuống chiếm khoảng: $T = (10.9 / 104.17) \times 100 \approx 10.46\%$ so với toàn bộ thời gian của một bit. Tỷ lệ này cho thấy quá trình chuyển mức từ cao xuống thấp diễn ra chậm hơn so với cạnh lý tưởng trong truyền thông UART. Mặc dù vẫn nằm trong ngưỡng cho phép ở tốc độ 9600 baud, việc cạnh xuống chiếm hơn 10% thời gian bit có thể làm giảm độ ổn định khi lấy mẫu, đặc biệt trong môi trường có nhiễu hoặc dao động tín hiệu. Điều này cho thấy tín hiệu UART US100 chưa thật sự tối ưu, và cần được cải thiện để đảm bảo độ chính xác cao hơn trong quá trình truyền dữ liệu.

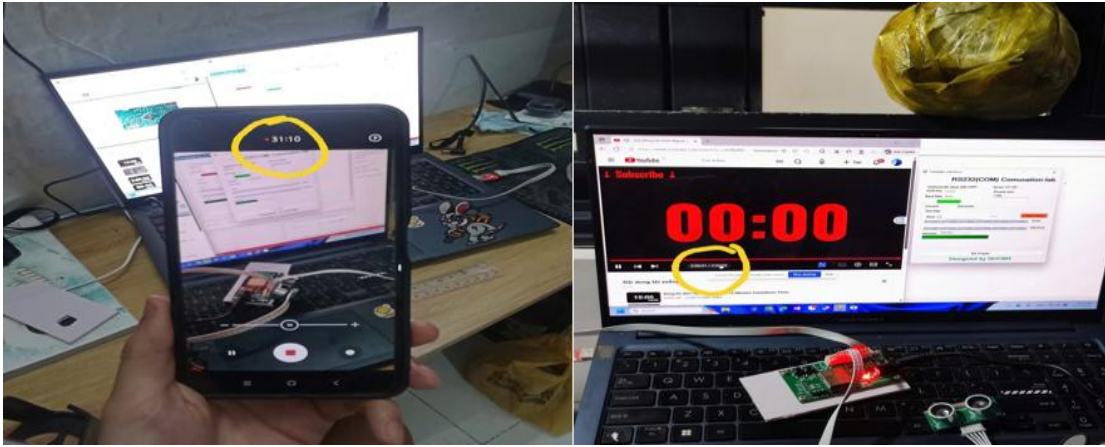
4.2.4 Thực nghiệm phần mềm và đánh giá

Ở phần 2.11 chúng em đã nêu khái quát về thực nghiệm phần mềm. Để kiểm tra chính xác nhóm đã tiến hành kiểm tra trên mô hình thực tế, thể hiện rõ tại hình 4.2.4.1. Kết quả cho thấy gửi 1000 kí tự nhận 1000 kí tự tỷ lệ đúng lệnh tới 100%. Một số lần có hiện tượng truyền sai 1-2 kí tự nhưng tỷ lệ lỗi vẫn nhỏ ($<0.2\%$) vẫn nằm trong mức cho phép. Điều này thể hiện Uart mềm hoạt động trơn tru đúng với yêu cầu mà nhóm đặt ra.



Hình 4.2.4.1: Minh chứng thực nghiệm phần mềm

Ngoài ra phần mềm chúng em còn có chức năng kiểm tra thủ công. Chức năng này cho phép nhập thủ công nhập bao nhiêu kí tự cũng được và ấn nút Send để bắt đầu quá trình kiểm thử. Sau đó trên màn hình sẽ xuất hiện bản kết quả test với các thông số để đánh giá.



Hình 4.2.5.1: Test liên tục 2h30p

Trong quá trình kiểm thử, dữ liệu từ ESP32-S3 được hiển thị theo thời gian thực, dễ quan sát và giao diện trực quan, không xảy ra hiện tượng giật hay đứng màn hình. Việc gửi và nhận dữ liệu qua UART diễn ra thành công, với sai số đo được khoảng 10%. Ứng dụng hoạt động ổn định, không bị treo khi truyền liên tục, đã thử nghiệm cả chiều gửi và phản hồi, kết quả vẫn duy trì sai số ở mức không qua 10%. Kết nối luôn ổn định, không bị mất kết nối bất thường, dữ liệu cập nhật đều đặn.

Giao diện được thiết kế thân thiện, trực quan, các nút chức năng hoạt động đúng như mong đợi, hiển thị rõ ràng và không gặp lỗi font chữ. Nhờ vậy, việc theo dõi và kiểm soát dữ liệu trở nên thuận tiện và dễ dàng hơn.

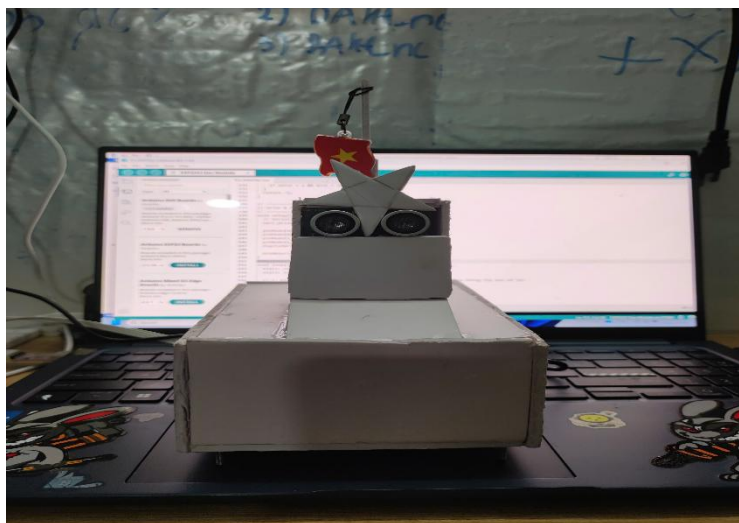
4.3 Lắp ráp mô hình

Sau khi đã kiểm tra và đảm bảo các chức năng hoạt động đúng theo yêu cầu đề ra, nhóm chúng em bắt đầu tiến hành lắp ráp mô hình. Từng bộ phận được chuẩn bị kỹ lưỡng, từ khung thân, cảm biến cho đến hệ thống điều khiển.



Hình 4.3.1: Cùng nhau lắp ráp mô hình

Mỗi thành viên đều tập trung cao độ, phối hợp nhịp nhàng để hoàn thiện từng bước và cùng nhau kiểm tra lại các chức năng chính. Sau 2 giờ nhóm đã hoàn thành mô hình và tất cả thành viên đều ân ý.



Hình 4.3.2: Mô hình thực tế sau khi lắp ráp

Mô hình nhìn chung tương đối đẹp và đặt biệt những món đồ để làm nên mô hình là những thứ nhóm đã có sẵn hầu như không mất tiền để mua.

CHƯƠNG 5: KẾT LUẬN VÀ ĐỀ XUẤT PHÁT TRIỂN

Sau quá trình nghiên cứu, thiết kế và triển khai, nhóm đã hoàn thiện hệ thống đo khoảng cách sử dụng cảm biến siêu âm US-100 kết hợp với vi điều khiển ESP32-S3. Hệ thống gồm đầy đủ các khối chức năng như mạch nguồn ổn định 3.3V, giao tiếp UART cứng và mềm, mạch cách ly tín hiệu bằng ADuM1201, cùng module CP2102 để truyền dữ liệu về máy tính. Phần mềm WinForms viết bằng C# giúp hiển thị kết quả đo theo thời gian thực, hỗ trợ người dùng theo dõi dễ dàng.

Thử nghiệm thực tế cho thấy hệ thống hoạt động ổn định, dữ liệu đo chính xác và hiển thị rõ ràng. Việc sử dụng mạch in hai lớp theo chuẩn công nghiệp giúp giảm nhiễu, tăng độ tin cậy và đảm bảo tính thẩm mỹ. Firmware được xây dựng bài bản, có lưu đồ rõ ràng, giúp hệ thống vận hành mượt mà và dễ mở rộng sau này. Nhìn chung, dự án đã đạt được mục tiêu đề ra, vừa mang tính ứng dụng cao, vừa mở ra hướng phát triển cho các hệ thống đo lường và giám sát dựa trên nền tảng IoT.

Tuy nhiên, hệ thống vẫn còn một số hạn chế. Việc sử dụng UART bị giới hạn bởi dây kết nối vật lý, gây khó khăn trong triển khai thực tế. Ngoài ra, hệ thống chưa có cơ sở dữ liệu để lưu trữ kết quả đo, cũng như chưa tích hợp chức năng hiển thị biểu đồ khoảng cách để người dùng dễ phân tích.

Trong giai đoạn tiếp theo, nhóm dự kiến sẽ nâng cấp hệ thống từ phần mềm WinForms C# sang ứng dụng di động, giúp người dùng dễ dàng theo dõi và điều khiển từ xa qua smartphone. Ứng dụng sẽ được tích hợp nền tảng cloud để lưu trữ dữ liệu đo lường, hỗ trợ phân tích và truy xuất lịch sử. Đồng thời, nhóm sẽ nghiên cứu bổ sung giao tiếp Zigbee để mở rộng khả năng kết nối không dây giữa các thiết bị, hướng đến xây dựng mô hình D2C (Device-to-Cloud) và D2D (Device-to-Device), phục vụ cho các hệ thống giám sát thông minh trong môi trường thực tế. Đây sẽ là bước tiến quan trọng giúp hệ thống trở nên linh hoạt, hiện đại và phù hợp hơn với các ứng dụng công nghiệp và dân dụng.

TÀI LIỆU THAM KHẢO

▪ Tài liệu tiếng Anh

[1] Robu.in, "US-100 Ultrasonic Sensor Module Datasheet," Robu.in, 2017.

[Online]. Available: https://robu.in/wpcontent/uploads/2017/09/US_100_ULTRASONIC_SENSOR_MODULE.pdf. [Accessed: Nov. 11, 2025].

[2] Texas Instruments, *LM1117T-3.3 Low-Dropout Linear Regulator Datasheet*,

2017. Available: <https://www.alldatasheet.com/datasheet-pdf/view/807165/TI1/LM1117T-3.3.html>

[3] Analog Devices, *ADuM1200/1201 Dual-Channel Digital Isolator Datasheet*, 2019.

Available: https://www.analog.com/media/en/technical-documentation/datasheets/ADuM1200_1201.pdf

[4] Espressif Systems, "ESP32-S3-WROOM-1 Datasheet," espressif.com, 2021. [Online]. Available: https://documentation.espressif.com/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf. [Accessed: Nov. 11, 2025].

[5] Microsoft, "Windows Forms Overview," learn.microsoft.com, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview>. [Accessed: Nov. 11, 2025].

[6] PlatformIO, "PlatformIO IDE for VS Code," platformio.org, 2023. [Online]. Available: <https://platformio.org/install/ide?install=vscode>. [Accessed: Nov. 11, 2025].

[7] SparkFun, "Serial Communication – UART," sparkfun.com, 2022. [Online]. Available: <https://learn.sparkfun.com/tutorials/serial-communication>. [Accessed: Nov. 11, 2025].

[8] ThegioiIC, "Giới thiệu về chuẩn giao tiếp UART," thegioiic.com, 2022. [Online]. Available: <https://www.thegioiic.com/tin-tuc/gioi-thieu-ve-chuan-giao-tiep-uart>. [Accessed: Nov. 12, 2025].

[8] Silicon Laboratories. *CP2102 USB to UART Bridge Controller Datasheet*. Truy cập tại: https://datasheet4u.com/pdf-down/C/P/2/CP2102_SiliconLaboratories.pdf

[9] WinForms Documentation, *Windows Forms Overview*, Microsoft Docs, 2023. Available: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>

[10] Dientutuonglai.com, *Tìm hiểu IC LM1117*, 2022. Available: <https://dientutuonglai.com/tim-hieu-lm1117.html>

[11] Espressif Systems, *ESP-IDF UART Programming Guide*, 2025. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html>

[12] Alex Chen, *The Ultimate Guide to Designing 2-Layer PCBs*, PCBASIC Blog, 2025. Available: https://www.pcbasic.com/blog/two_layer_pcb.html

[13] *Mạch chuyển USB to TTL CP2102*. Truy cập tại: <https://nshopvn.com/product/mach-chuyen-usb-to-ttl-cp2102>

- Các trang Web

[1] Website: <http://www.dientuvietnam.net>

[2] Website: <https://nshopvn.com>